SQL interview question:

Create a Database in SQL

Explore this section to get hands on all the cheat sheet that help you in order to create a database in SQL.

1. CREATE DATABASE: Create a New Database

CREATE DATABASE company;

This command creates a new database named "**company**."

2. USE: Select a Specific Database to Work With

USE company;

This command selects the database named "**company**" for further operations.

3. ALTER DATABASE: Modify a Database's Attributes

ALTER DATABASE database_name

4. DROP DATABASE: Delete an Existing Database

DROP DATABASE company;

Creating Data in SQL

Here in this SQL cheat sheet we have listed down all the cheat sheet that help to create, insert, alter data in table.

5. CREATE: Create a New Table, Database or Index

```
CREATE TABLE employees (
  employee_id INT PRIMARY KEY,
  first_name VARCHAR(50),
  last_name VARCHAR(50),
  department VARCHAR(50),
  salary DECIMAL(10, 2)
);
```

This command creates a table named "employees" with columns for employee ID, first name, last name, department, and salary. The employee_id column is set as the primary key.

6. INSERT INTO: Add New Records To A Table

```
INSERT INTO employees (employee_id, first_name, last_name, department, salary)
VALUES
  (1, 'John', 'Doe', 'HR', 50000.00),
  (2, 'Jane', 'Smith', 'IT', 60000.00),
  (3, 'Alice', 'Johnson', 'Finance', 55000.00),
  (4, 'Bob', 'Williams', 'IT', 62000.00),
  (5, 'Emily', 'Brown', 'HR', 48000.00);
```

This command inserts sample data into the "employees" table with values for **employee ID, first name, last name, department, and salary.**

7. ALTER TABLE: Modify An Existing Table's Structure

ALTER TABLE employees ADD COLUMN new_column INT;

This command adds a new column named "new_column" of integer type to the existing "employees" table.

8. DROP TABLE: Delete A Table And Its Data

DROP TABLE employees;

This command deletes the entire "**employees**" table along with all its data.

Reading/Querying Data in SQL

Explore this section to get the cheat sheet on how to use select, distinct and other querying data in SQL.

9. SELECT: Retrieve Data From One Or More Tables

SELECT * FROM employees;

This query will retrieve all columns from the employees table.

10. DISTINCT: Select Unique Values From A Column

SELECT DISTINCT department FROM employees;

This query will return unique department names from the employees table.

11. WHERE: Filter Rows Based On Specified Conditions

SELECT * FROM employees WHERE salary > 55000.00;

This query will return employees whose salary is greater than 55000.00.

12. LIMIT: Limit The Number Of Rows Returned In The Result Set

SELECT * FROM employees LIMIT 3;

This query will limit the result set to the first 3 rows.

13. OFFSET: Skip A Specified Number Of Rows Before Returning The Result Set

SELECT * FROM employees LIMIT 10000 OFFSET 2;

This query retrieves all rows from the "employees" table, skipping the first 2 rows and limiting the result to 10,000 rows.

14. FETCH: Retrieve A Specified Number Of Rows From The Result Set

SELECT * FROM employees FETCH FIRST 3 ROWS ONLY;

This query will fetch the first 3 rows from the result set.

15. CASE: Perform Conditional Logic In A Query

```
SELECT
  first_name,
  last_name,
  CASE
    WHEN salary > 55000 THEN 'High'
    WHEN salary > 50000 THEN 'Medium'
    ELSE 'Low'
  END AS salary_category
FROM employees;
```

Updating/Manipulating Data in SQL

Get a cheat sheet on how to update or manipulate data in SQL by exploring this section.

16. UPDATE: Modify Existing Records In A Table

```
UPDATE employees
SET salary = 55000.00
WHERE employee_id = 1;
```

This query will update the salary of the employee with employee_id 1 to 55000.00.

Deleting Data in SQL

17. DELETE: Remove Records From A Table

```
DELETE FROM employees
WHERE employee_id = 5;
```

This query will delete the record of the employee with employee_id 5 from the employees table.

Filtering Data in SQL

18. WHERE: Filter Rows Based On Specified Conditions

```
SELECT * FROM employees
WHERE department = 'IT';
```

This query will retrieve all employees who work in the IT department.

19. LIKE: Match A Pattern In A Column

```
SELECT * FROM employees
WHERE first_name LIKE 'J%';
```

This query will retrieve all employees whose first name starts with 'J'.

20. IN: Match Any Value In A List

```
SELECT * FROM employees
WHERE department IN ('HR', 'Finance');
```

This query will retrieve all employees who work in the HR or Finance departments.

21. BETWEEN: Match Values Within A Specified Range

```
SELECT * FROM employees
WHERE salary BETWEEN 50000 AND 60000;
```

This query will retrieve all employees whose salary is between 50000 and 60000.

22. IS NULL: Match NULL Values

```
SELECT * FROM employees
WHERE department IS NULL;
```

This query will retrieve all employees where the department is not assigned (NULL).

23. ORDER BY: Sort The Result Set

```
SELECT * FROM employees
ORDER BY salary DESC;
```

This query will retrieve all employees sorted by salary in descending order.

<u>SQL Operator:</u>

Here in this section we have added a cheat sheet for SQL Operators. So, explore and learn how to use AND, OR, NOT and others oprtators.

24. AND: Combines Multiple Conditions In A WHERE Clause

```
SELECT * FROM employees
WHERE department = 'IT' AND salary > 60000;
```

This query will retrieve employees who work in the IT department and have a salary greater than 60000.

25. OR: Specifies Multiple Conditions Where Any One Of Them Should Be True

```
SELECT * FROM employees
WHERE department = 'HR' OR department = 'Finance';
```

This query will retrieve employees who work in either the HR or Finance department.

26. NOT: Negates A Condition

```
SELECT * FROM employees
WHERE NOT department = 'IT';
```

This query will retrieve employees who do not work in the IT department.

27. LIKE: Searches For A Specified Pattern In A Column

SELECT * FROM employees WHERE first_name LIKE 'J%';

This query will retrieve employees whose first name starts with '**J**'.

28. IN: Checks If A Value Matches Any Value In

SELECT * FROM employees WHERE department IN ('HR', 'Finance');

This query will retrieve employees who work in the HR or Finance departments.

29. BETWEEN: Selects Values Within a Specified Range

SELECT * FROM employees WHERE salary BETWEEN 50000 AND 60000;

This query will retrieve employees whose salary is between 50000 and 60000.

30. IS NULL: Checks if a Value is NULL

SELECT * FROM employees WHERE department IS NULL;

This query will retrieve employees where the department is not assigned (NULL).

31. ORDER BY: Sorts the Result Set in Ascending or Descending Order

SELECT * FROM employees ORDER BY salary DESC;

This query will retrieve all employees sorted by salary in descending order.

32. GROUP BY: Groups Rows that have the Same Values into Summary Rows

SELECT department, COUNT(*) AS employee_count FROM employees GROUP BY department;

This query will group employees by department and count the number of employees in each department

Aggregation Data in SQL

Get an hands in aggregation data in SQL. Here you will find cheat sheet for how to count numbers, sum of numbers and more.

33. COUNT: Count The Number Of Rows In A Result Set

SELECT COUNT(*) FROM employees;

This query will count the total number of employees.

34. SUM: Calculate The Sum Of Values In A Column

SELECT SUM(salary) FROM employees;

This query will calculate the total salary of all employees.

35. AVG: Calculate The Average Value Of A Column

SELECT AVG(salary) FROM employees;

This query will calculate the average salary of all employees.

36. MIN: Find the Minimum Value in a Column

SELECT MIN(salary) FROM employees;

This query will find the minimum salary among all employees.

37. MAX: Find the Maximum Value in a Column

SELECT MAX(salary) FROM employees;

This query will find the maximum salary among all employees.

38. GROUP BY: Group Rows Based on a Specified Column

```
SELECT department, COUNT(*) AS employee_count
FROM employees
GROUP BY department;
```

This query will group employees by department and count the number of employees in each department.

39. HAVING: Filter Groups Based on Specified Conditions

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 55000;
```

This query will calculate the average salary for each department and return only those departments where the average salary is greater than 55000.

Constraints in SQL:

Constraints in SQL act as data quality guardrails, enforcing rules to ensure accuracy, consistency, and integrity within your database tables.

40. PRIMARY KEY: Uniquely Identifies Each Record in a Table

```
CREATE TABLE employees (
  employee_id INT PRIMARY KEY,
  first_name VARCHAR(50),
  last_name VARCHAR(50)
);
```

employee_id is designated as the primary key, ensuring that each employee record has a unique identifier.

41. FOREIGN KEY: Establishes a Relationship Between Two Tables

```
CREATE TABLE departments (
  department_id INT PRIMARY KEY,
  department_name VARCHAR(50)
);
```

```
CREATE TABLE employees (
  employee_id INT PRIMARY KEY,
  first_name VARCHAR(50),
  last_name VARCHAR(50),
  department_id INT,
  FOREIGN KEY (department_id) REFERENCES departments(department_id)
);
```

department_id column in the employees table is a foreign key that references the department_id column in the departments table, establishing a relationship between the two tables.

42. UNIQUE: Ensures That All Values in a Column Are Unique

```
CREATE TABLE employees (
  employee_id INT PRIMARY KEY,
  email VARCHAR(100) UNIQUE
);
```

email column must contain unique values for each employee.

43. NOT NULL: Ensures That a Column Does Not Contain NULL Values

```
CREATE TABLE employees (
  employee_id INT PRIMARY KEY,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL
);
```

first_name and **last_name** columns must have values and cannot be NULL.

44. CHECK: Specifies a Condition That Must Be Met for a Column's Value

```
CREATE TABLE employees (
  employee_id INT PRIMARY KEY,
  age INT CHECK (age >= 18)
);
```

age column must have a value of 18 or greater due to the CHECK constraint.

Joins in SQL

Explore different join types to seamlessly merge data from multiple tables in your SQL queries.

45. INNER JOIN: Retrieves Records That Have Matching Values in Both Tables

```
SELECT * FROM employees
INNER JOIN departments ON employees.department_id = departments.department_id;
```

This query will retrieve records from both the employees and departments tables where there is a match on the department_id column.

46. LEFT JOIN: Retrieves All Records from the Left Table and the Matched Records from the Right Table

```
SELECT * FROM employees
LEFT JOIN departments ON employees.department_id = departments.department_id;
```

This query will retrieve all records from the employees table and only the matching records from the departments table.

47. RIGHT JOIN: Retrieves All Records from the Right Table and the Matched Records from the Left Table

```
SELECT * FROM employees
RIGHT JOIN departments ON employees.department_id = departments.department_id;
```

This query will retrieve all records from the departments table and only the matching records from the employees table.

48. FULL OUTER JOIN: Retrieves All Records When There Is a Match in Either the Left or Right Table

```
SELECT * FROM employees
FULL OUTER JOIN departments ON employees.department_id =
departments.department_id;
```

This query will retrieve all records from both the employees and departments tables, including unmatched records.

49. CROSS JOIN: Retrieves the Cartesian Product of the Two Tables

```
SELECT * FROM employees
CROSS JOIN departments;
```

This query will retrieve all possible combinations of records from the employees and departments tables.

50. SELF JOIN: Joins a Table to Itself

```
SELECT e1.first_name, e2.first_name
FROM employees e1, employees e2
WHERE e1.employee_id = e2.manager_id;
```

In this example, the employees table is joined to itself to find employees and their respective managers based on the manager_id column.

SQL Functions

In this section we have compiled SQL cheat sheet for SQL functions. It is used for common tasks like aggregation, filtering, date/time manipulation, and more!

51. Scalar Functions: Functions That Return a Single Value

```
SELECT UPPER(first_name) AS upper_case_name FROM employees;
```

This query uses the UPPER() scalar function to convert the first_name column values to uppercase.

52. Aggregate Functions: Functions That Operate on a Set of Values and Return a Single Value

```
SELECT AVG(salary) AS average_salary FROM employees;
```

SQL Functions

In this section we have compiled SQL cheat sheet for SQL functions. It is used for common tasks like aggregation, filtering, date/time manipulation, and more!

51. Scalar Functions: Functions That Return a Single Value

SELECT UPPER(first_name) AS upper_case_name FROM employees;

This query uses the UPPER() scalar function to convert the first_name column values to uppercase.

52. Aggregate Functions: Functions That Operate on a Set of Values and Return a Single Value

SELECT AVG(salary) AS average_salary FROM employees;

This query uses the AVG() aggregate function to calculate the average salary of all employees.

53. String Functions: Functions That Manipulate String Values

SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM employees;

This query uses the CONCAT() string function to concatenate the first_name and last_name columns into a single column called full_name.

SELECT SUBSTR(first_name, 1, 3) AS short_name FROM employees;

This query uses the SUBSTR() function to extract the first three characters of the first_name column for each employee. The result is displayed in a new column called short_name.

SELECT INSERT(full_name, 6, 0, 'Amazing ') AS modified_name
FROM (SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM employees) AS employee_names;

This query first concatenates the first_name and last_name columns into a single column called full_name. Then, it uses the INSERT() function to insert the string 'Amazing ' at the 6th position of the full_name column for each employee. The modified names are displayed in a new column called modified_name.

54. Date and Time Functions: Functions That Operate on Date and Time Values

SELECT CURRENT_DATE AS current_date FROM dual;

This query uses the CURRENT_DATE date function to retrieve the current date.

55. Mathematical Functions: Functions That Perform Mathematical Operations

SELECT SQRT(25) AS square_root FROM dual;

This query uses the SQRT() mathematical function to calculate the square root of 25.

Subqueries in SQL

This SQL cheat sheet explains how to nest queries for powerful data filtering and manipulation within a single statement.

56. Single-row Subquery: Returns One Row of Result

```
SELECT first_name, last_name
FROM employees
WHERE salary = (SELECT MAX(salary) FROM employees);
```

In this example, the subquery (SELECT MAX(salary) **FROM** employees) returns a single row containing the maximum salary, and it's used to filter employees who have the maximum salary.

57. Multiple-row Subquery: Returns Multiple Rows of Result

```
SELECT department_name
FROM departments
WHERE department_id IN (SELECT department_id FROM employees);
```

In this example, the subquery (SELECT department_id FROM employees) returns multiple rows containing department IDs, and it's used to filter department names based on those IDs.

58. Correlated Subquery: References a Column from the Outer Query

```
SELECT first_name, last_name
FROM employees e
WHERE salary > (SELECT AVG(salary) FROM employees WHERE department =
e.department);
```

In this example, the subquery (SELECT **AVG**(salary) FROM employees WHERE department = e.department) is correlated with the outer query by referencing the department column from the outer query. It calculates the average salary for each department and is used to filter employees whose salary is greater than the average salary of their respective department.

59. Nested Subquery: A Subquery Inside Another Subquery

```
SELECT first_name, last_name
FROM employees
WHERE department_id IN (
   SELECT department_id
   FROM departments
   WHERE department_name = 'IT'
);
```

In this example, the subquery (SELECT department_id FROM departments WHERE department_name = 'IT') is nested within the outer query. It retrieves the department ID for the IT department, which is then used in the outer query to filter employees belonging to the IT department.

Views in SQL

Here in this SQL cheat sheet unveils how to create virtual tables based on existing data for streamlined access.

60. CREATE VIEW: Create a Virtual Table Based on the Result of a SELECT Query

```
CREATE VIEW high_paid_employees AS
SELECT *
FROM employees
WHERE salary > 60000;
```

This query creates a views named high_paid_employees that contains all employees with a salary greater than 60000.

61. DROP VIEW: Delete a View

```
DROP VIEW IF EXISTS high_paid_employees;
```

This query drops the high_paid_employees view if it exists.

Indexes in SQL

Speed up your SQL queries with our Indexes Cheat Sheet! Learn how to create and optimize indexes to dramatically improve database performance.

62. CREATE INDEX: Create an Index on a Table

```
CREATE INDEX idx_department ON employees (department);
```

This query creates an index named idx_department on the department column of the employees table.

63. DROP INDEX: Remove an Index

```
DROP INDEX IF EXISTS idx_department;
```

This query drops the idx_department index if it exists.

SQL Views

Views in SQL are a type of **virtual table** that simplifies how users interact with data across one or more tables. Unlike **traditional tables**, a view in **SQL** does not store data on disk; instead, it dynamically retrieves data based on a pre-defined query each time it's accessed.

SQL views are particularly useful for managing complex queries, enhancing security, and presenting data in a simplified format. In this guide, we will cover the **SQL** create view statement, updating and deleting views, and using the WITH CHECK OPTION clause.

What is a View in SQL?

A view in SQL is a saved SQL query that acts as a virtual table. It can fetch data from one or more tables and present it in a customized format, allowing developers to:

- **Simplify Complex Queries:** Encapsulate complex joins and conditions into a single object.
- **Enhance Security:** Restrict access to specific columns or rows.
- **Present Data Flexibly:** Provide tailored data views for different users.

We will be using these **two SQL tables** for examples.

StudentDetails

```
-- Create StudentDetails table
CREATE TABLE StudentDetails (
   S_ID INT PRIMARY KEY,
   NAME VARCHAR(255),
   ADDRESS VARCHAR(255)
);

INSERT INTO StudentDetails (S_ID, NAME, ADDRESS)
VALUES
   (1, 'Harsh', 'Kolkata'),
   (2, 'Ashish', 'Durgapur'),
   (3, 'Pratik', 'Delhi'),
   (4, 'Dhanraj', 'Bihar'),
   (5, 'Ram', 'Rajasthan');
```

S_ID	NAME	ADDRESS
1	Harsh	Kolkata
2	Ashish	Durgapur
3	Pratik	Delhi
4	Dhanraj	Bihar
5	Ram	Rajasthan

StudentMarks

```
-- Create StudentMarks table
CREATE TABLE StudentMarks (
   ID INT PRIMARY KEY,
   NAME VARCHAR(255),
   Marks INT,
   Age INT
);

INSERT INTO StudentMarks (ID, NAME, Marks, Age)
VALUES
   (1, 'Harsh', 90, 19),
   (2, 'Suresh', 50, 20),
   (3, 'Pratik', 80, 19),
   (4, 'Dhanraj', 95, 21),
   (5, 'Ram', 85, 18);
```

ID	NAME	MARKS	AGE
1	Harsh	90	19
2	Suresh	50	20
3	Pratik	80	19
4	Dhanraj	95	21
5	Ram	85	18

CREATE VIEWS in SQL

We can create a view using **CREATE VIEW** statement. A View can be created from a single table or multiple tables.

Syntax:

CREATE VIEW view_name AS SELECT column1, column2….. FROM table_name

WHERE condition;

Parameters:

- **view_name**: Name for the View
- **table_name**: Name of the table
- **condition**: Condition to select rows

SQL CREATE VIEW Statement Examples

Let's look at some examples of CREATE VIEW Statement in SQL to get a better understanding of how to create views in SQL.

Example 1: Creating View From a Single Table

In this example, we will create a View named DetailsView from the table StudentDetails.
Query:

CREATE VIEW DetailsView **AS**

SELECT NAME, ADDRESS

FROM StudentDetails

WHERE S_ID < 5;

To see the data in the View, we can query the view in the same manner as we query a table.

SELECT * **FROM** DetailsView;

Output:

NAME	ADDRESS
Harsh	Kolkata
Ashish	Durgapur
Pratik	Delhi
Dhanraj	Bihar

Example 2: Create View From Table

In this example, we will create a view named StudentNames from the table StudentDetails.
Query:

CREATE VIEW StudentNames **AS**

SELECT S_ID, NAME

FROM StudentDetails

ORDER BY NAME;

If we now query the view as,

SELECT * **FROM** StudentNames;

Output:

S_ID	NAMES
2	Ashish
4	Dhanraj
1	Harsh
3	Pratik
5	Ram

Example 3: Creating View From Multiple Tables

In this example we will create a View named MarksView from two tables StudentDetails and StudentMarks. To create a View from multiple tables we can simply include multiple tables in the SELECT statement. Query:

CREATE VIEW MarksView AS

SELECT StudentDetails.NAME, StudentDetails.ADDRESS, StudentMarks.MARKS

FROM StudentDetails, StudentMarks

WHERE StudentDetails.NAME = StudentMarks.NAME;

To display data of View MarksView:

SELECT * FROM MarksView;

Output:

NAME	ADDRESS	MARKS
Harsh	Kolkata	90
Pratik	Delhi	80
Dhanraj	Bihar	95
Ram	Rajasthan	85

Listing all Views in a Database

We can list View using the **SHOW FULL TABLES** statement or using the **information_schema table**. A View can be created from a single table or multiple tables.

Syntax:

USE "database_name";

SHOW FULL TABLES **WHERE** table_type LIKE "%VIEW";

Using information_schema

SELECT table_name

FROM information_schema.views

WHERE table_schema = 'database_name';

- Retrieves only the names of the views (table_name) in the specified database (database_name).
- Useful if you just need a list of all views in the database
- If you just need to list all views, use the **first query**.

OR

SELECT table_schema, table_name, view_definition

FROM information_schema.views

WHERE table_schema = 'database_name';

- table_schema: The database name.
- table_name: The name of the view.
- view_definition: The SQL statement that defines the view.
- If you need to examine how views are defined, use the **second query**.

Delete View in SQL

SQL allows us to delete an existing View. We can delete or drop View using the **DROP statement**.

Syntax:

DROP VIEW view_name;

Example

In this example, we are deleting the View **MarksView.**

DROP VIEW MarksView;

Update View in SQL

If you want to update the existing data within the view, use the **UPDATE** statement.

Syntax:

UPDATE view_name

SET column1 = value1, column2 = value2...., columnN = valueN

WHERE [condition];

Note: Not all views can be updated using the UPDATE statement.

If you want to update the view definition without affecting the data, use the **CREATE OR REPLACE VIEW** statement. you can use this syntax

CREATE OR REPLACE VIEW view_name **AS**

SELECT column1, column2, ...

FROM table_name

WHERE condition;

SQL CREATE VIEW Statement Examples

Let's look at some examples of CREATE VIEW Statement in SQL to get a better understanding of how to create views in SQL.

Example 1: Creating View From a Single Table

In this example, we will create a View named DetailsView from the table StudentDetails.
Query:

CREATE VIEW DetailsView **AS**

SELECT NAME, ADDRESS

FROM StudentDetails

WHERE S_ID < 5;

To see the data in the View, we can query the view in the same manner as we query a table.

SELECT * FROM DetailsView;

Output:

NAME	ADDRESS
Harsh	Kolkata
Ashish	Durgapur
Pratik	Delhi
Dhanraj	Bihar

Example 2: Create View From Table

In this example, we will create a view named StudentNames from the table StudentDetails. Query:

CREATE VIEW StudentNames **AS**

SELECT S_ID, NAME

FROM StudentDetails

ORDER BY NAME;

If we now query the view as,

SELECT * FROM StudentNames;

Output:

S_ID	NAMES
2	Ashish
4	Dhanraj
1	Harsh
3	Pratik
5	Ram

Example 3: Creating View From Multiple Tables

In this example we will create a View named MarksView from two tables StudentDetails and StudentMarks. To create a View from multiple tables we can simply include multiple tables in the SELECT statement. Query:

CREATE VIEW MarksView AS

SELECT StudentDetails.NAME, StudentDetails.ADDRESS, StudentMarks.MARKS

FROM StudentDetails, StudentMarks

WHERE StudentDetails.NAME = StudentMarks.NAME;

To display data of View MarksView:

SELECT * FROM MarksView;

Output:

NAME	ADDRESS	MARKS
Harsh	Kolkata	90
Pratik	Delhi	80
Dhanraj	Bihar	95
Ram	Rajasthan	85

Listing all Views in a Database

We can list View using the **SHOW FULL TABLES** statement or using the **information_schema table**. A View can be created from a single table or multiple tables.

Syntax:

USE "database_name";

SHOW FULL TABLES **WHERE** table_type LIKE "%VIEW";

Using information_schema

SELECT table_name

FROM information_schema.views

WHERE table_schema = 'database_name';

OR

SELECT table_schema, table_name, view_definition

FROM information_schema.views

WHERE table_schema = 'database_name';

Delete View in SQL

SQL allows us to delete an existing View. We can delete or drop View using the **DROP statement**.

Syntax:

DROP VIEW view_name;

Example

In this example, we are deleting the View **MarksView.**

DROP VIEW MarksView;

Update View in SQL

If you want to update the existing data within the view, use the **UPDATE** statement.

Syntax:

UPDATE view_name

SET column1 = value1, column2 = value2...., columnN = valueN

WHERE [condition];

Note: Not all views can be updated using the UPDATE statement.

If you want to update the view definition without affecting the data, use the **CREATE OR REPLACE VIEW** statement. you can use this syntax

CREATE OR REPLACE VIEW view_name **AS**

SELECT column1, column2, ...

FROM table_name

WHERE condition;

Rules to Update Views in SQL:

Certain conditions need to be satisfied to update a view. If any of these conditions are **not** met, the view can not be updated.

1. The SELECT statement which is used to create the view should not include GROUP BY clause or ORDER BY clause.
2. The SELECT statement should not have the DISTINCT keyword.
3. The View should have all NOT NULL values.
4. The view should not be created using nested queries or complex queries.
5. The view should be created from a single table. If the view is created using multiple tables then we will not be allowed to update the view.

Examples

Let's look at different use cases for updating a view in SQL. We will cover these use cases with examples to get a better understanding.

Example 1: Update View to Add or Replace a View Field

We can use the **CREATE OR REPLACE VIEW** statement to add or replace fields from a view.

If we want to update the view **MarksView** and add the field AGE to this View from **StudentMarks** Table, we can do this by:

CREATE OR REPLACE VIEW MarksView AS

SELECT StudentDetails.NAME, StudentDetails.ADDRESS, StudentMarks.MARKS, StudentMarks.AGE

FROM StudentDetails, StudentMarks

WHERE StudentDetails.NAME = StudentMarks.NAME;

If we fetch all the data from MarksView now as:

SELECT * FROM MarksView;

Output:

NAME	ADDRESS	MARKS	AGE
Harsh	Kolkata	90	19
Pratik	Delhi	80	19
Dhanraj	Bihar	95	21
Ram	Rajasthan	85	18

Example 2: Update View to Insert a row in a view

We can insert a row in a View in the same way as we do in a table. We can use the INSERT INTO statement of SQL to insert a row in a View.

In the below example, we will insert a new row in the View DetailsView which we have created above in the example of "creating views from a single table".

INSERT INTO DetailsView(NAME, ADDRESS)

VALUES("Suresh","Gurgaon");

If we fetch all the data from DetailsView now as,

SELECT * FROM DetailsView;

Output:

NAME	ADDRESS
Harsh	Kolkata
Ashish	Durgapur
Pratik	Delhi
Dhanraj	Bihar
Suresh	Gurgaon

Example 3: Deleting a row from a View

Deleting rows from a view is also as simple as deleting rows from a table. We can use the DELETE statement of SQL to delete rows from a view. Also deleting a row from a view first deletes the row from the actual table and the change is then reflected in the view.

In this example, we will delete the last row from the view DetailsView which we just added in the above example of inserting rows.

DELETE FROM DetailsView

WHERE NAME="Suresh";

If we fetch all the data from DetailsView now as,

SELECT * FROM DetailsView;

Output:

NAME	ADDRESS
Harsh	Kolkata
Ashish	Durgapur
Pratik	Delhi
Dhanraj	Bihar

WITH CHECK OPTION Clause

The **WITH CHECK OPTION** clause in SQL is a very useful clause for views. It applies to an updatable view.

The WITH CHECK OPTION clause is used to prevent data modification (using INSERT or UPDATE) if the condition in the WHERE clause in the CREATE VIEW statement is not satisfied.

If we have used the WITH CHECK OPTION clause in the CREATE VIEW statement, and if the UPDATE or INSERT clause does not satisfy the conditions then they will return an error.

WITH CHECK OPTION Clause Example:

In the below example, we are creating a View SampleView from the StudentDetails Table with a WITH CHECK OPTION clause.

CREATE VIEW SampleView AS

SELECT S_ID, NAME

FROM StudentDetails

WHERE NAME IS NOT NULL

WITH CHECK OPTION;

In this view, if we now try to insert a new row with a null value in the NAME column then it will give an error because the view is created with the condition for the NAME column as NOT NULL. For example, though the View is updatable then also the below query for this View is not valid:

INSERT INTO SampleView(S_ID)

VALUES(6);

***NOTE**: The default value of NAME column is null.*

Uses of a View

A good database should contain views for the given reasons:

1. **Restricting data access –** Views provide an additional level of table security by restricting access to a predetermined set of rows and columns of a table.
2. **Hiding data complexity –** A view can hide the complexity that exists in multiple joined tables.
3. **Simplify commands for the user –** Views allow the user to select information from multiple tables without requiring the users to actually know how to perform a join.
4. **Store complex queries –** Views can be used to store complex queries.
5. **Rename Columns –** Views can also be used to rename the columns without affecting the base tables provided the number of columns in view must match the number of columns specified in a select statement. Thus, renaming helps to hide the names of the columns of the base tables.
6. **Multiple view facility –** Different views can be created on the same table for different users.

Key Takeaways About SQL Views

- *Views in SQL are a kind of virtual table.*
- *The fields in a view can be from one or multiple tables.*
- *We can create a view using the CREATE VIEW statement and delete a view using the DROP VIEW statement.*
- *We can update a view using the CREATE OR REPLACE VIEW statement.*
- *WITH CHECK OPTION clause is used to prevent inserting new rows that do not satisfy the view's filtering condition.*
- `DROP INDEX IF EXISTS idx_department;`
- This query drops the idx_department index if it exists.
- Transactions in SQL
- Learn how to manage groups of database operations as a single unit for reliable data updates.
- 64. BEGIN TRANSACTION: Start a New Transaction
- `BEGIN TRANSACTION;`
- This statement starts a new transaction.

- 65. COMMIT: Save Changes Made During the Current Transaction
- `COMMIT;`
- This statement saves all changes made during the current transaction.
- 66. ROLLBACK: Undo Changes Made During the Current Transaction
- `ROLLBACK;`
- This statement undoes all changes made during the current transaction.

SQL TRANSACTIONS

SQL transactions are essential for ensuring **data integrity** and consistency in **relational databases**. Transactions allow for a group of **SQL operations** to be executed as a **single unit**, ensuring that either all the operations succeed or none of them do. This capability, together with the **ACID** (Atomicity, Consistency, Isolation, and Durability) properties, is fundamental for maintaining the **integrity of our data.**

What is an SQL Transaction?

A transaction in SQL is a sequence of one or more SQL statements executed as a **single unit of work**. These statements could be performing operations like **INSERT**, UPDATE, or **DELETE**. The main idea behind a transaction is that all changes made within it should either be completely successful or fully undone in case of failure. In the case of an error, a **rollback** ensures that no partial changes are saved.

The SQL transaction must adhere to four key properties known as **ACID**:

- **Atomicity:** The outcome of a transaction can either be completely successful or completely unsuccessful. The whole transaction must be rolled back if one part of it fails.
- **Consistency:** Transactions maintain integrity restrictions by moving the database from one valid state to another.
- **Isolation:** Concurrent transactions are isolated from one another, assuring the accuracy of the data.
- **Durability:** Once a transaction is committed, its modifications remain in effect even in the event of a system failure.

The SQL Transaction Process

Transactions are controlled using specific SQL commands. The most common commands used for managing transactions are:

1. **BEGIN TRANSACTION**: Starts a transaction.
2. **COMMIT**: Commits the transaction, saving all changes made.
3. **ROLLBACK**: Rolls back the transaction, undoing all changes made since the BEGIN TRANSACTION.
4. **SAVEPOINT**: Marks a point within a transaction to which you can later roll back.

Example of SQL Transaction with a Bank Transfer Scenario

Let's look at an example of a bank transfer between two accounts. This example demonstrates the usage of multiple queries in a single transaction.

```
BEGIN TRANSACTION;

-- Deduct $150 from Account A
UPDATE Accounts
SET Balance = Balance - 150
WHERE AccountID = 'A';

-- Add $150 to Account B
UPDATE Accounts
SET Balance = Balance + 150
WHERE AccountID = 'B';

-- Commit the transaction if both operations succeed
COMMIT;
```

If any error occurs, such as an issue with the UPDATE query, you can use ROLLBACK to undo all changes made during the transaction:

```
ROLLBACK;
```

This ensures that the system doesn't end up in an inconsistent state, such as deducting money from one account without adding it to another.

SQL Transaction Control Commands

In SQL, transaction control commands manage the **execution of SQL operations**, ensuring the integrity and reliability of **database transactions**. These commands help manage the start, commit, and rollback of changes made to the database. Below are the **key transaction control commands** in SQL, explained with syntax and examples for each.

1. BEGIN TRANSACTION Command

The **BEGIN TRANSACTION** command marks the beginning of a new transaction. All SQL statements that follow this command will be part of the same transaction until a **COMMIT** or ROLLBACK is encountered. This command doesn't make any changes to the database, it just starts the transaction.

Syntax:

BEGIN TRANSACTION transaction_name ;

Example

BEGIN TRANSACTION TransferFunds;

2. COMMIT Command

The COMMIT command is used to save all changes made during the current transaction to the database. Once a transaction is committed, the changes are permanent.

Syntax:

COMMIT;

Here is the sample **Student** table that will be used to perform the operations in this example. This table contains basic student details such as **ID**, **name**, **age**, and other relevant information that will be manipulated using various **transaction control commands**.

Student				
Rol_No	Name	Address	Phone	Age
1	Ram	Delhi	9455123451	18
2	Ramesh	Gurgaon	9652431543	18
3	Sujit	Rohtak	9156253131	20
4	Suresh	Delhi	9156768971	18
3	Sujit	Rohtak	9156253131	20
2	Ramesh	Gurgaon	9652431543	18

Student Table

Example

Following is an example which would delete those records from the table which have age = 20 and then COMMIT the changes in the database.

```
DELETE FROM Student WHERE AGE = 20;
COMMIT;
```

Output

Rol_No	Name	Address	Phone	Age
1	Ram	Delhi	9455123451	18
2	Ramesh	Gurgaon	9652431543	18
4	Suresh	Delhi	9156768971	18
2	Ramesh	Gurgaon	9652431543	18

output

3. ROLLBACK Command

The **ROLLBACK command** is used to undo all changes made in the current transaction. It is used when an error occurs or when the desired changes cannot be completed. The database will

revert to the state it was in before the BEGIN TRANSACTION was executed.

Syntax:

ROLLBACK;

Example

Delete those records from the table which have age = 20 and then ROLLBACK the changes in the database. In this case, the DELETE operation is undone, and the changes to the database are not saved.

DELETE FROM Student WHERE AGE = 20;
ROLLBACK;

Output

Student				
Rol_No	Name	Address	Phone	Age
1	Ram	Delhi	9455123451	18
2	Ramesh	Gurgaon	9652431543	18
3	Sujit	Rohtak	9156253131	20
4	Suresh	Delhi	9156768971	18
3	Sujit	Rohtak	9156253131	20
2	Ramesh	Gurgaon	9652431543	18

output

4. SAVEPOINT Command

A SAVEPOINT is used to create a checkpoint within a transaction. We can roll back to a specific SAVEPOINT instead

of rolling back the entire transaction. This allows us to **undo part** of the transaction **rather than** the entire transaction.

Syntax:

SAVEPOINT SAVEPOINT_NAME;

Example

SAVEPOINT SP1;
//Savepoint created.
DELETE FROM Student WHERE AGE = 20;
//deleted

SAVEPOINT SP2;
//Savepoint created.

Output

Rol_No	Name	Address	Phone	Age
1	Ram	Delhi	9455123451	18
2	Ramesh	Gurgaon	9652431543	18
4	Suresh	Delhi	9156768971	18
2	Ramesh	Gurgaon	9652431543	18

output

Explanation:

From the above example **Sample table1**, Delete those records from the table which have age = 20 and then ROLLBACK the changes in the database by keeping Savepoints. Here SP1 is first SAVEPOINT created before deletion. In this example one

deletion have taken place. After deletion again SAVEPOINT SP2 is created.

5. ROLLBACK TO SAVEPOINT

The **ROLLBACK TO SAVEPOINT** command allows us to roll back the transaction to a specific savepoint, effectively undoing changes made after that point.

Syntax:

ROLLBACK TO SAVEPOINT SAVEPOINT_NAME;

Example

Deletion have been taken place, let us assume that we have changed our mind and decided to ROLLBACK to the SAVEPOINT that we identified as SP1 which is before deletion. So, In this case the DELETE operation is undone, and the transaction is returned to the state it was in at the SP1 savepoint.

ROLLBACK TO SP1;
//Rollback completed

Output

Student				
Rol_No	Name	Address	Phone	Age
1	Ram	Delhi	9455123451	18
2	Ramesh	Gurgaon	9652431543	18
3	Sujit	Rohtak	9156253131	20
4	Suresh	Delhi	9156768971	18
3	Sujit	Rohtak	9156253131	20
2	Ramesh	Gurgaon	9652431543	18

output

6. RELEASE SAVEPOINT Command

This command is used to remove a **SAVEPOINT** that we have created. Once a SAVEPOINT has been released, we can no longer use the **ROLLBACK command** to undo transactions performed since the last SAVEPOINT. It is used to initiate a **database transaction** and used to specify characteristics of the transaction that follows.

Syntax:

RELEASE SAVEPOINT SAVEPOINT_NAME

Example

Once the savepoint SP2 is released, we can no longer roll back to it.

RELEASE SAVEPOINT SP2; -- Release the second savepoint.

Summary of Transaction Control Commands

Command	Purpose	Example
BEGIN TRANSACTION	Starts a new transaction.	BEGIN TRANSACTION TransferFunds;
COMMIT	Saves all changes made during the transaction.	COMMIT;
ROLLBACK	Undoes changes made during the transaction.	ROLLBACK;
SAVEPOINT	Creates a point in the transaction to which you can roll back.	SAVEPOINT SP1;
ROLLBACK TO SAVEPOINT	Rolls back the transaction to a specific savepoint.	ROLLBACK TO SP1;
RELEASE SAVEPOINT	Removes a savepoint from the transaction.	RELEASE SAVEPOINT SP2;

Types of SQL Transactions

There are different types of transactions based on their nature and the specific operations they perform:

- **Read Transactions**: Used to only read the data, typically with SELECT queries.
- **Write Transactions**: These involve modifying the data in the database with INSERT, UPDATE, or DELETE operations.
- **Distributed Transactions**: These transactions span multiple databases and ensure consistency across them.
- **Implicit Transactions**: Automatically started by SQL Server for certain operations.
- **Explicit Transactions**: Manually controlled transactions where the user begins and ends the transaction using BEGIN TRANSACTION, COMMIT, and ROLLBACK.

How to Monitor and Optimize SQL Transactions

Optimizing SQL transactions is critical for performance. Here are some best practices:

- **Keep Transactions Short**: Avoid long-running transactions as they can lock tables and reduce concurrency.
- **Use Proper Indexing**: Proper indexing on tables involved in the transaction can greatly improve performance.
- **Avoid Transaction Blocking**: Ensure transactions don't block others unnecessarily, causing delays.
- **Commit Early**: Try to commit as soon as the transaction is complete to release resources.

Conclusion

SQL transactions are a fundamental part of **database management**, providing a way to ensure **data consistency**, integrity, and isolation. Understanding how to use

BEGIN TRANSACTION, **COMMIT**, **ROLLBACK**, and SAVEPOINT is essential for writing reliable and efficient SQL queries. With proper transaction control, we can ensure that our database operations are **robust**, **consistent**, and optimized for performance. By following best practices like keeping transactions

short and committing early, we can also improve the **scalability** and concurrency of our database.

FAQs

What is the SQL transaction?

An SQL transaction is a sequence of SQL statements executed as a single unit of work. It ensures that either all operations are successfully completed or none of them are, maintaining the database's integrity using properties like ACID (Atomicity, Consistency, Isolation, Durability).

What is Transact SQL with an example?

Transact-SQL (T-SQL) is an extension of SQL used in Microsoft SQL Server. It provides additional features such as procedural programming, error handling, and control-of-flow constructs.

Example:

```
BEGIN TRANSACTION;
UPDATE Employees SET Salary = 60000 WHERE EmployeeID = 1;
COMMIT;
```

What is the difference between a query and a transaction in SQL?

A query is a single SQL statement used to retrieve or modify data in the database, while a transaction is a series of queries that are executed together to ensure data consistency, with support for commit or rollback in case of errors.

Advanced Mixed Data in SQL

In the last we have complied all the important queries under the one advanced SQL cheat sheet.

67. Stored Procedures: Precompiled SQL Statements That Can Be Executed with a Single Command

```
CREATE PROCEDURE get_employee_count()
BEGIN
  SELECT COUNT(*) FROM employees;
END;
```

SQL Stored Procedures *are a powerful feature in* ***database management systems*** *(DBMS) that allow developers to encapsulate SQL code and business logic. Stored procedures are precompiled SQL statements that are stored in the* ***database*** *and can be executed as a* ***single unit****.*

What is a SQL Stored Procedure?

A SQL Stored Procedure is a collection of SQL statements ***bundled together*** *to perform a specific task. These procedures are stored in the database and can be called upon by users, applications, or other procedures. Stored procedures are essential for* ***automating database tasks****, improving efficiency, and* ***reducing redundancy****. By encapsulating logic within* ***stored procedures****, developers can streamline their* ***workflow*** *and enforce* ***consistent business rules*** *across multiple applications and systems.*

Syntax:

```
CREATE PROCEDURE procedure_name
(parameter1 data_type, parameter2 data_type, ...)
AS
BEGIN
  — SQL statements to be executed
END
```

Key Terms

- *CREATE PROCEDURE: This keyword creates the stored procedure with the given name.*
- *@parameter1, @parameter2: These are input parameters that allow you to pass values into the stored procedure.*
- *BEGIN...END: These keywords define the block of SQL statements that make up the procedure body.*

Why Use SQL Stored Procedures?

1. ***Performance Optimization**: Since stored procedures are precompiled, they execute faster than running ad-hoc SQL queries. The database engine can reuse the execution plan, eliminating the need for repeated query parsing and optimization.*

2. ***Security**: By using stored procedures, developers can restrict direct access to sensitive data. Users can execute procedures without accessing the underlying tables, helping to protect critical information.*

3. ***Code Reusability**: SQL stored procedures can be reused in multiple applications or different parts of an application. This reduces the need to rewrite complex queries repeatedly.*

4. ***Reduced Network Traffic**: Instead of sending multiple individual queries to the database server, stored procedures allow you to execute multiple operations in one go, reducing network load.*

5. ***Maintainability**: Stored procedures simplify code maintenance. Changes made to the procedure are automatically reflected wherever the procedure is used, making it easier to manage complex logic.*

By passing a country as a parameter, the stored procedure dynamically fetches the relevant customer details from the table----------

Create a stored procedure named "GetCustomersByCountry"

```
CREATE PROCEDURE GetCustomersByCountry
   @Country VARCHAR(50)
AS
BEGIN
   SELECT CustomerName, ContactName
   FROM Customers
   WHERE Country = @Country;
END;
```

-- Execute the stored procedure with parameter "Sri Lanka"

EXEC GetCustomersByCountry @Country = 'Sri lanka';

***A stored procedure** in SQL is a set of precompiled SQL statements that can be executed as a single unit. It is stored in the database and can be invoked to perform various tasks, such as data manipulation, executing queries, and automating repetitive operations.*

Types of SQL Stored Procedures

SQL stored procedures are categorized into different types based on their use case and functionality:

1. ***System Stored Procedures**: These are predefined stored procedures provided by the SQL Server for performing administrative tasks. Examples include sp_help for viewing database object information and sp_rename for renaming database objects.*

2. ***User-Defined Stored Procedures**: These are custom stored procedures created by the user to perform specific operations. For example, creating a procedure that calculates the total sales for a particular product category.*

3. ***Extended Stored Procedures**: These allow for the execution of external functions, which might be implemented in other languages such as C or C++. Extended procedures are typically used for integrating external tools into SQL Server.*

4. ***CLR Stored Procedures**: These are stored procedures written in .NET languages (like C#) and executed within SQL Server. CLR stored procedures are useful when advanced functionality is needed that isn't easily achievable with T-SQL alone*

68. Triggers: Automatically Execute a Set of SQL Statements When a Specified Event Occurs

```
CREATE TRIGGER before_employee_insert
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
  SET NEW.creation_date = NOW();
END;
```

SQL triggers are a critical feature in **database management systems (DBMS)** that provide automatic execution of a set of SQL statements when specific database events, such as **INSERT**, **UPDATE**, or **DELETE** operations, occur. Triggers are commonly used to maintain **data integrity**, **track changes**, and **enforce business rules** automatically, without needing manual input.

What is SQL Trigger?

A trigger is a stored procedure in a **database** that automatically invokes whenever a special event in the database occurs. By using **SQL triggers**, developers can **automate tasks**, ensure **data consistency**, and keep accurate records of **database activities**. For example, a trigger can be invoked when a row is inserted into a specified table or when specific table columns are updated.

In simple words, a **trigger** is a collection of SQL statements with particular names that are stored in system memory. It belongs to a specific class of **stored procedures** that are automatically invoked in response to database server events. Every **trigger** has a table attached to it.

Key Features of SQL Triggers:

- **Automatic Execution**: Triggers fire automatically when the defined event occurs (e.g., INSERT, UPDATE, DELETE).
- **Event-Driven**: Triggers are tied to specific events that take place within the database.
- **Table Association**: A trigger is linked to a specific table or view, and operates whenever changes are made to the table's data.

Syntax

```
create trigger [trigger_name]
[before | after]
{insert | update | delete}
on [table_name]
FOR EACH ROW
BEGIN

//sql statement body
END;
```

Key Terms

- **trigger_name:** The name of the trigger to be created.
- **BEFORE | AFTER:** Specifies whether the trigger is fired **before** or **after** the triggering event (INSERT, UPDATE, DELETE).
- **{INSERT | UPDATE | DELETE}:** Specifies the operation that will activate the trigger.
- **table_name:** The name of the table the trigger is associated with.
- **FOR EACH ROW:** Indicates that the trigger is row-level, meaning it executes once for each affected row.
- **trigger_body:** The SQL statements to be executed when the trigger is fired.

Types of SQL Triggers

Triggers can be categorized into **different types** based on the action they are associated with:

1. DDL Triggers

The **Data Definition Language** (DDL) command events such as **Create_table**, **Create_view**, **drop_table**, **Drop_view**,and **Alter_table**

cause the DDL triggers to be activated. They allow us to track changes in the structure of the database. The trigger will prevent any table creation, alteration, or deletion in the database.

Example:

```
CREATE TRIGGER prevent_table_creation
ON DATABASE
FOR CREATE_TABLE, ALTER_TABLE, DROP_TABLE
AS
BEGIN
   PRINT 'you can not create, drop and alter table in this
database';
   ROLLBACK;
END;
```

2. DML Triggers

The Data manipulation Language (DML) command events that begin with **Insert**, **Update**, and **Delete** set off the DML triggers. DML triggers are used for data validation, ensuring that modifications to a table are done under controlled conditions.

Example:

```
CREATE TRIGGER prevent_update
ON students
FOR UPDATE
AS
BEGIN
  PRINT 'You can not insert, update and delete this table i';
  ROLLBACK;
END;
```

3. Logon Triggers

These triggers are fired in response to **logon events**. Logon triggers are useful for **monitoring user sessions** or restricting user access to the database. As a result, the **PRINT statement** messages and any errors generated by the trigger will all be visible in the **SQL Server** error log. **Authentication errors** prevent logon triggers from being used. These triggers can be used to **track login activity** or set a limit on the number of sessions that a given login can have in order to **audit** and **manage server sessions**.

Example:

```
CREATE TRIGGER track_logon
ON LOGON
AS
BEGIN
  PRINT 'A new user has logged in.';
END;
```

Using SQL Triggers: Practical Use Cases

1. Automatically Updating Related Tables (DML Trigger Example)

Triggers can automatically perform tasks, like **updating related tables** when data changes. For instance, when a student's grade is updated, the total score in the related table can also be updated automatically.

Example:

```
CREATE TRIGGER update_student_score
AFTER UPDATE ON student_grades
FOR EACH ROW
BEGIN
  UPDATE total_scores
  SET score = score + :new.grade
  WHERE student_id = :new.student_id;
END;
```

2. Data Validation (Before Insert Trigger Example)

Triggers can be used to validate data before it is inserted into a table, ensuring that the data follows specific business rules.

Example:

```
CREATE TRIGGER validate_grade
BEFORE INSERT ON student_grades
FOR EACH ROW
BEGIN
  IF :new.grade < 0 OR :new.grade > 100 THEN
    RAISE_APPLICATION_ERROR(-20001, 'Invalid grade value.');
  END IF;
END;
```

Output

This trigger ensures that no grade less than 0 or greater than 100 is inserted into the student_grades table.

Viewing Triggers in SQL

*If we are working with many tables across multiple **databases**, we can use a simple query to list all available triggers in our SQL Server **instance**. This is helpful for **tracking** and **managing triggers**, especially when dealing with tables that have similar names across **databases**.*

Syntax

```
SELECT name, is_instead_of_trigger
FROM sys.triggers
WHERE type = 'TR';
```

Key Terms

- ***name**: The name of the trigger.*
- ***is_instead_of_trigger**: Whether the trigger is an **INSTEAD OF** trigger.*
- ***type = 'TR'**: This filters the results to show only triggers.*

*The SQL Server Management Studio makes it very simple to display or **list all triggers** that are available for any given table. The following steps will help us accomplish this:*

*Go to the **Databases** menu, select the desired database, and then expand it.*

- *Select the **Tables** menu and expand it.*
- *Select any specific table and expand it.*
- *We will get various options here. When we choose the **Triggers** option, it displays all the triggers available in this table.*

BEFORE and AFTER Triggers

*SQL triggers can be specified to run **BEFORE** or **AFTER** the triggering event.*

- ***BEFORE Triggers:** Execute before the actual SQL statement is executed. Useful for validating data before insertion or updating.*
- ***AFTER Triggers:** Execute after the SQL statement completes. Useful for logging or cascading updates to other tables.*

Example: Using BEFORE Trigger for Calculations

*Given **Student Report Database**, in which student marks **assessment** is recorded. In such a schema, create a trigger so that the **total** and **percentage** of specified marks are automatically inserted whenever a record is inserted. Here, a **trigger will invoke** before the record is inserted so **BEFORE Tag** can be used.*

Query

mysql>>desc Student;

```
+-------+-------------+------+-----+---------+----------------+
| Field | Type        | Null | Key | Default | Extra          |
+-------+-------------+------+-----+---------+----------------+
| tid   | int(4)      | NO   | PRI | NULL    | auto_increment |
| name  | varchar(30) | YES  |     | NULL    |                |
| subj1 | int(2)      | YES  |     | NULL    |                |
| subj2 | int(2)      | YES  |     | NULL    |                |
| subj3 | int(2)      | YES  |     | NULL    |                |
| total | int(3)      | YES  |     | NULL    |                |
| per   | int(3)      | YES  |     | NULL    |                |
+-------+-------------+------+-----+---------+----------------+
```

Student Datatype

*Below **SQL statement** will create a trigger in the **student database** in which whenever subjects marks are entered, before inserting this data into*

*the **database**, the trigger will compute those **two values** and insert them with the entered values. In this way, triggers can be **created** and **executed** in the **databases**.*

```
CREATE TRIGGER stud_marks
BEFORE INSERT ON Student
FOR EACH ROW
SET NEW.total = NEW.subj1 + NEW.subj2 + NEW.subj3,
    NEW.per = (NEW.subj1 + NEW.subj2 + NEW.subj3) * 60 / 100;
```

Stud_marks

Output

```
+------------+--------+-----------+--------------+---------------------+----
| Trigger    | Event  | Table     | Timing       | Created             | Stat
+------------+--------+-----------+--------------+---------------------+----
| stud_marks | BEFORE | Student   | BEFORE INSERT | 2023-05-02 00:00:00 | ACT
+------------+--------+-----------+--------------+---------------------+----
```

Output

Advantages of Triggers

- ***Data Integrity**: Triggers help enforce consistency and business rules, ensuring that data follows the correct format.*
- ***Automation**: Triggers eliminate the need for manual intervention by automatically performing tasks such as updating, inserting, or deleting records when certain conditions are met.*
- ***Audit Trail**: Triggers can track changes in a database, providing an audit trail of **INSERT**, UPDATE, and **DELETE** operations.*
- ***Performance**: By automating repetitive tasks, triggers improve **SQL query performance** and reduce manual workload.*

FAQs

What is an SQL trigger?

An SQL trigger is a special type of stored procedure that automatically executes or fires when certain events occur in a database, such as INSERT, UPDATE, or DELETE on a table.

How do SQL triggers work?

SQL triggers work by being automatically invoked in response to specific actions (like data modifications) on a table or view, based on the event, timing (before or after), and condition specified in the trigger definition.

What are the benefits of using SQL triggers?

SQL triggers help ensure data integrity, enforce business rules, and automatically audit changes in the database. They reduce the need for repetitive manual tasks and improve consistency and security in database operations.

69. User-defined Functions (UDFs): Custom SQL Functions Created by Users to Perform Specific Tasks

```
CREATE FUNCTION calculate_bonus(salary DECIMAL) RETURNS
DECIMAL
BEGIN
  RETURN salary * 0.1; -- 10% bonus
END;
```

This query creates a user-defined function named calculate_bonus that calculates the bonus based on the salary.

Scenario:

*A company wants to calculate **performance-based bonuses** for its employees. The bonus is **10% of the salary** by default, but if an employee has more than **5 years of experience**, the bonus increases to **15%**.*

```
DELIMITER $$

CREATE FUNCTION calculate_bonus(salary DECIMAL(10,2),
years_of_experience INT)

RETURNS DECIMAL(10,2)

DETERMINISTIC

BEGIN

  DECLARE bonus DECIMAL(10,2);

  IF years_of_experience > 5

 THEN

    SET bonus = salary * 0.15; -- 15% bonus for experienced employees

 ELSE

    SET bonus = salary * 0.10; -- 10% bonus for others

END IF;

  RETURN bonus;

END $$

DELIMITER ;
```

2 Using the UDF in a Query

Suppose we have an Employees table with columns EmployeeID, Name, Salary, and YearsOfExperience.
We can use our calculate_bonus() function to determine the bonus for each employee.

```
SELECT EmployeeID, Name, Salary, YearsOfExperience,
calculate_bonus(Salary, YearsOfExperience) AS  Bonusn FROM Employees
```

Example 1: Calculate Tax Based on Salary Slabs

Scenario: *A company wants to calculate* ***income tax*** *based on salary slabs:*

- ***Salary ≤ 50,000 → 5% tax***
- ***Salary between 50,001 and 100,000 → 10% tax***
- ***Salary > 100,000 → 15% tax***

```
DELIMITER $$

CREATE FUNCTION calculate_tax(salary DECIMAL(10,2))

RETURNS DECIMAL(10,2) DETERMINISTIC

BEGIN

        DECLARE tax DECIMAL(10,2);

         IF salary <= 50000    THEN

                SET tax = salary * 0.05;

         ELSEIF salary <= 100000 THEN

                SET tax = salary * 0.10;

        ELSE

               SET tax = salary * 0.15;

        END IF;

     RETURN tax;

END

$$ DELIMITER ;
```

Using the Function in a Query

SELECT EmployeeID, Name, Salary, calculate_tax(Salary) AS Tax FROM Employees;

Example 2: Calculate Discount Based on Purchase Amount

Scenario: An **e-commerce platform** wants to offer discounts based on **total purchase value**: **DECIMAL(10,2)** means: 10 → Maximum total digits (including before and after decimal).2 → Maximum digits after the decimal point.1)**< 5000** → No discount, 2)**5,000 - 10,000 → 5% discount,**

3)10,001 - 20,000 → 10% discount,4)> 20,000 → 15% discount

```
DELIMITER $$

CREATE FUNCTION calculate_discount(amount DECIMAL(10,2))

RETURNS DECIMAL(10,2) DETERMINISTIC

BEGIN

   DECLARE discount DECIMAL(10,2);

   IF amount < 5000 THEN

          SET discount = 0;

  ELSEIF    amount BETWEEN  5000   AND   10000  THEN

        SET discount = amount * 0.05;

  ELSEIF  amount  BETWEEN  10001  AND   20000 THEN

        SET discount = amount * 0.10;

   ELSE

      SET discount = amount * 0.15;

  END IF;

  RETURN discount;

END

$$DELIMITER ;
```

Using the Function in a Query

SELECT OrderID, CustomerName, TotalAmount, calculate_discount(TotalAmount) AS Discount FROM Orders;

Example 3: Calculate Student Grade Based on Marks

Scenario:A school wants to assign grades to students based on their marks:

- **Marks ≥ 90 → Grade A**
- **Marks 80-89 → Grade B**
- **Marks 70-79 → Grade C**
- **Marks 60-69 → Grade D**
- **Marks < 60 → Grade F**

```
DELIMITER $$

CREATE FUNCTION get_grade(marks INT)

RETURNS CHAR(1)  DETERMINISTIC

BEGIN

      DECLARE grade CHAR(1);

      IF marks >= 90 THEN

           SET grade = 'A';

    ELSEIF marks >= 80 THEN

          SET grade = 'B';

    ELSEIF marks >= 70 THEN

         SET grade = 'C';

   ELSEIF marks >= 60 THEN

        SET grade = 'D';

   ELSE

       SET grade = 'F';

 END IF;
```

```
    RETURN grade;

END

$$ DELIMITER ;
```

Using the Function in a Query

```
SELECT StudentID, Name, Marks, get_grade(Marks) AS Grade FROM Students
```

---

70. Common Table Expressions (CTEs): Temporary Result Sets That Can Be Referenced Within a SELECT, INSERT, UPDATE, or DELETE Statement

```
WITH high_paid_employees AS (
   SELECT * FROM employees WHERE salary > 60000
)
SELECT * FROM high_paid_employees;
```

This query uses a common table expression named high_paid_employees to retrieve all employees with a salary greater than 60000.

In SQL, a **Common Table Expression (CTE)** is an essential tool for simplifying **complex queries** and making them more readable. By defining temporary result sets that can be referenced multiple times, a **CTE in SQL** allows developers to break down **complicated logic** into manageable parts. CTEs help with **hierarchical data representation**, improve code reusability, and simplify maintenance

Why do We Need CTE in the SQL ?

A **Common Table Expression (CTE)** in SQL is a temporary result set that is defined and used within the execution scope of a **SELECT**, **INSERT**, **UPDATE**,**or DELETE** statement. CTEs are defined using the WITH clause and can be referenced multiple times within the main SQL query. This makes CTEs a great alternative to **subqueries**, especially in cases where we need to perform the same operation multiple times or create **recursive queries**.

Why Use CTEs in SQL?

CTEs simplify query writing and maintenance by:

- Breaking down complex queries into smaller, reusable components.
- Improving code readability and modularity.
- Enabling recursive operations for hierarchical data.

Syntax

```
WITH  cte_name  AS (
SELECT query
)
SELECT * FROM cte_name;
```

Key Terms

- **cte_name**: A unique name for the CTE expression.
- **query**: A valid SQL query that returns a result set, which will be treated as a virtual table within the main query.
- **SELECT**: The main query that can reference the CTE by its name.

Creating a Simple CTE in SQL

Let's consider an **Employees** table that contains employee details such as **EmployeeID**, **Name**, **Department**, **Salary**, and **ManagerID**. This table is used to demonstrate how to use a **Common Table Expression (CTE)** to simplify SQL queries, particularly when aggregating or filtering data.

Name	Age	Department	Salary
Ramesh	20	Finance	50,000
Suresh	22	Finance	50,000
Ram	28	Finance	20,000
Deep	25	Sales	30,000
Pradeep	22	Sales	20,000

Employees table

Example: Calculate Average Salary by Department

In this example, we will use a **Common Table Expression (CTE)** to calculate the average salary for each department in the **Employees** table. The CTE simplifies the query by breaking it into a manageable part that can be referenced in the main query.

Query:

```
WITH AvgSalaryByDept AS (
    SELECT Department, AVG(Salary) AS AvgSalary FROM Employees
    GROUP BY Department
)
SELECT * FROM AvgSalaryByDept;
```

Output

Department	AvgSalary
IT	75000
HR	60000
Finance	52500

Explanation:

- The **WITH** clause defines a CTE named AvgSalaryByDept.
- The main query references this CTE to retrieve the average salary for each department.

Recursive Common Table Expression (RCTE)

*A recursive CTE is one that references itself within that CTE. The recursive **CTE** is useful when working with hierarchical data as the CTE continues to execute until the query returns the entire hierarchy.*

*A typical example of hierarchical data is a table that includes a list of **employees**. For every employee, the table provides a reference to that person's manager. That reference is itself an employee ID within the same table. We can use a r**ecursive CTE** to display the hierarchy of employee data.*

*If a CTE is created incorrectly it can enter an infinite loop. To prevent this, the **MAXRECURSION** hint can be added in the OPTION clause of the primary SELECT, INSERT, UPDATE, DELETE, or **MERGE statement.***

Example: Hierarchical Employee Data

```
WITH  cteReports (EmpID, FirstName, LastName, MgrID, EmpLevel)
 AS
 (
   SELECT EmployeeID, FirstName, LastName, ManagerID, 1
   FROM Employees
   WHERE ManagerID IS NULL


   UNION ALL


   SELECT e.EmployeeID, e.FirstName, e.LastName, e.ManagerID,
     r.EmpLevel + 1
   FROM Employees e
     INNER JOIN cteReports r
       ON e.ManagerID = r.EmpID
 )

SELECT
 FirstName + ' ' + LastName AS FullName,
 EmpLevel, (SELECT FirstName + ' ' + LastName FROM Employees
 WHERE EmployeeID = cteReports.MgrID) AS Manager FROM cteReports
 ORDER BY EmpLevel, MgrID
```

Output

Name	***Level***
John Smith	*1*
Jane Doe	*2*
Alice Brown	*2*
Bob Green	*3*
Charlie Ray	*3*

📊 *Sample Data & Output Example*

🔹 *Employees Table*

EmployeeID	***FirstName***	***LastName***	***ManagerID***
1	*Alice*	*Smith*	*NULL*
2	*Bob*	*Johnson*	*1*
3	*Charlie*	*Brown*	*1*
4	*David*	*Lee*	*2*
5	*Emma*	*Wilson*	*2*
6	*Frank*	*Taylor*	*3*

🔹 *Query Output*

FullName	***EmpLevel***	***Manager***
Alice Smith	*1*	*NULL*
Bob Johnson	*2*	*Alice Smith*
Charlie Brown	*2*	*Alice Smith*
David Lee	*3*	*Bob Johnson*
Emma Wilson	*3*	*Bob Johnson*
Frank Taylor	*3*	*Charlie Brown*

Explanation:

- *The **anchor member** selects employees with no manager (ManagerID IS NULL), establishing the base level.*

- *The **recursive member** joins the CTE to the table, incrementing the level for each subordinate employee.*

Benefits of Using CTEs in SQL

1. ***Improved Readability**: CTEs help break down complex queries into modular, reusable components, improving code readability and maintainability.*

2. ***Reusability**: Once defined, a CTE can be referenced multiple times within the same query, reducing the need for repetitive code.*

3. ***Simplifies Complex Queries**: By using CTEs, especially recursive CTEs, complex operations like hierarchical data queries become much easier to manage.*

4. ***Query Optimization**: SQL engines can optimize queries that use CTEs more efficiently, improving performance, especially when the same result set needs to be accessed multiple times.*

Limitations of CTEs in SQL

- ***Temporary Scope**: A CTE exists only during the execution of the query. Once the query completes, the CTE is discarded.*

- ***Performance Issues**: For very large datasets, CTEs can sometimes lead to performance degradation due to multiple references to the same CTE.*

- ***Not Allowed in All Database Operations**: Some operations, such as **INSERT** and UPDATE, may have restrictions when using CTEs in certain databases.*

- ***Common Table Expressions (CTEs)** are a valuable feature in SQL, enhancing query readability, **modularity**, and performance. By using CTEs, developers can simplify **complex queries**, especially those involving **hierarchical data** or repeated subqueries.*

CTE vs Subqueries

Feature	CTE	Subquery
Reusability	Can be referenced multiple times.	Typically used once.
Readability	Improves readability for complex queries.	Can become difficult to read when nested.
Performance	Optimized for multiple references.	May be less efficient for repeated operations.

FAQs

What is a CTE in SQL?

A CTE (Common Table Expression) in SQL is a temporary result set defined within the execution scope of a SELECT, INSERT, UPDATE, or DELETE statement. It simplifies complex queries by allowing you to break them into reusable subqueries that improve readability and maintainability.

What is CTE in SQL interview questions?

In SQL interviews, a CTE is commonly discussed to assess a candidate's understanding ***of query optimization and the ability to handle complex querie ,the use of CTEs for breaking down subqueries, improving performance, and organizing SQL code effectively.***

What is the difference between CTE and subquery?

A ***CTE*** *is* ***defined before the main query and can be referenced multiple times****, improving readability and reusability.* ***A subquery, on the other hand, is embedded within a query and can only be referenced once****. CTEs tend to*

be easier to debug and maintain compared to subqueries, especially in complex SQL operations.

Window Functions in SQL:

SQL window functions *are essential for advanced* ***data analysis*** *and* ***database management****.* ***They enable calculations across a specific set of rows, known as a "window," while retaining the individual rows in the dataset****.* *Unlike traditional* ***aggregate functions*** *that summarize data for the entire group, window functions allow detailed calculations for specific partitions or* ***subsets of data****.*

This article provides a detailed explanation of ***SQL window functions****, including the* ***OVER clause****,* ***partitioning****,* ***ordering****, and practical use cases, complete with outputs to help us understand their behaviour.*

What is a Window Function in SQL?

A ***window function*** *in SQL is a type of function that allows us to perform* ***calculations*** *across a specific set of rows related to the current row. These calculations happen within a defined* ***window of data****, and they are particularly useful for* ***aggregates****,* ***rankings****, and* ***cumulative totals*** *without altering the dataset.*

The OVER clause is key to defining this window. It partitions the data into different sets (using the ***PARTITION BY*** *clause) and orders them (using the* ***ORDER BY*** *clause). These windows enable functions like* ***SUM()****,* ***AVG()****,* ***ROW_NUMBER()****,* ***RANK()****, and* ***DENSE_RANK()*** *to be applied in a sophisticated manner.*

Syntax *:*

```
SELECT column_name1, window_function (column_name2)
OVER ([PARTITION BY column_name1] [ORDER BY column_name3])
AS new_column  FROM  table_name;
```

Key Terms

- ***window_function****= any aggregate or ranking function*
- ***column_name1****= column to be selected*
- ***column_name2****= column on which window function is to be applied*

- ***column_name3***= *column on whose basis partition of rows is to be done*
- ***new_column***= *Name of new column*
- ***table_name***= *Name of table*

Types of Window Functions in SQL

SQL window functions can be categorized into two primary types: **aggregate window functions** and **ranking window functions**. These two types serve different purposes but share a common ability to perform calculations over a defined **set of rows** while retaining the original data. The **employee** table contains details about employees, such as their name, age, department, and salary.

employees Table

Name	Age	Department	Salary
Ramesh	20	Finance	50,000
Suresh	22	Finance	50,000
Ram	28	Finance	20,000
Deep	25	Sales	30,000
Pradeep	22	Sales	20,000

1. Aggregate Window Function

Aggregate window functions calculate aggregates over a window of rows while retaining individual rows. These include **SUM()**, AVG(), **COUNT()**, **MAX()**, and **MIN()**.

Example: AVG() Function

Query to calculate the average salary within each department:

```
SELECT Name, Age, Department, Salary,
AVG(Salary) OVER( PARTITION BY Department) AS Avg_Salary
FROM employee
```

Output

Name	Age	Department	Salary	Avg_Salary
Ramesh	20	Finance	50,000	40,000
Suresh	22	Finance	50,000	40,000
Ram	28	Finance	20,000	40,000
Deep	25	Sales	30,000	25,000
Pradeep	22	Sales	20,000	25,000

AVG() Function Example

Explanation:

- The **AVG()** function calculates the average salary for each department using the **PARTITION BY Department** clause.
- The average salary is repeated for all rows in the respective department.

2. Ranking Window Functions

These functions provide rankings of rows within a partition based on specific criteria. Common ranking functions include:

1. RANK() Function

The RANK() function assigns ranks to rows within a partition, with the same rank given to rows with identical values. If two rows share the same rank, the next rank is skipped.

Example:

```
SELECT Name, Department, Salary,
RANK() OVER(PARTITION BY Department ORDER BY Salary DESC AS
emp_rank  FROM employee;
```

Output

Name	Department	Salary	emp_rank
Ramesh	Finance	50,000	1
Suresh	Finance	50,000	1
Ram	Finance	20,000	3
Deep	Sales	30,000	1
Pradeep	Sales	20,000	2

Rank Function Example

Explanation:

- Rows with the same salary (e.g., Ramesh and Suresh) are assigned the same rank.
- The next rank is skipped (e.g., rank 2) due to duplicate ranks.

2. DENSE_RANK() Function

It assigns rank to each row within partition. Just like rank function first row is assigned rank 1 and rows having same value have same rank. The difference between RANK() and DENSE_RANK() is that in DENSE_RANK(), **for the next rank after two same rank, consecutive integer is used, no rank is skipped.**

Example:

```
SELECT Name, Department, Salary,
DENSE_RANK() OVER(PARTITION BY Department ORDER BY Salary DESC) AS
emp_dense_rank  FROM employee;
```

Output

Name	Department	Salary	emp_dense_rank
Ramesh	Finance	50,000	1
Suresh	Finance	50,000	1
Ram	Finance	20,000	2
Deep	Sales	30,000	1
Pradeep	Sales	20,000	2

3. ROW_NUMBER() Function

ROW_NUMBER() gives each row a **unique number**. It numbers rows from one to the total rows. The rows are put into **groups** based on their values. Each group is called a **partition**. **In each partition, rows get numbers one after another. No two rows have the same number in a partition.**

Example:

```
SELECT Name, Department, Salary,
ROW_NUMBER() OVER(PARTITION BY Department ORDER BY Salary
DESC) AS  emp_row_no  FROM employee;
```

Output

Name	Department	Salary	emp_row_no
Ramesh	Finance	50,000	1
Suresh	Finance	50,000	2
Ram	Finance	20,000	3
Deep	Sales	30,000	1
Pradeep	Sales	20,000	2

Definition of Window Function in SQL

A **window function** in SQL **performs a calculation across a specific range of rows (called a window) that are related to the current row**, without collapsing the result into a single output row.

Unlike **aggregate functions** (SUM(), AVG(), etc.), which group multiple rows into one, window functions **retain individual rows while adding computed values based on a defined window of data**.

General Syntex:

<window_function>() OVER ([PARTITION BY column_name]

[ORDER BY column_name])

FAQs

What is a window function in SQL?

A window function in SQL performs a calculation across a set of table rows related to the current row within a specified window. Unlike regular aggregate functions, window functions allow you to retain individual rows while performing the calculation.

1⃣ Aggregate Window Functions

*Perform **aggregations across a defined window of rows**.*

Function	***Description***
SUM()	*Returns **cumulative sum** over a window.*
AVG()	*Returns **moving average** over a window.*
MIN() / MAX()	*Returns **min/max within a window**.*
COUNT()	*Returns **count of rows** within a window.*

📌 ***Example:*** *Calculate **total, average, and highest salary per department**.*

```
SELECT EmployeeID, FirstName, Salary, Department,
    SUM(Salary) OVER (PARTITION BY Department) AS TotalSalary,
    AVG(Salary) OVER (PARTITION BY Department) AS AvgSalary,
    MAX(Salary) OVER (PARTITION BY Department) AS MaxSalary
FROM EmployeeSalaries;
```

What is the window function OVER in SQL?

The OVER clause in SQL defines the window or range of rows the window function should operate on. It is used with functions like ROW_NUMBER(), RANK(), or SUM() to calculate values across a partition of the result set.

What is the window function SQL LAST?

The LAST_VALUE() function in SQL is a window function that returns the last value in a specified window of rows. It allows you to retrieve the last value within a partition, ordered according to the given sorting criteria.

4.Offset (Lag/Lead) Functions

*Retrieve **values from previous or next rows**.*

Function	***Description***
LAG()=	*Gets **value from the previous row***
LEAD()=	*Gets **value from the next row***

Step 1: Creating the Table

```
CREATE TABLE EmployeeSalaries (

   EmployeeID INT PRIMARY KEY,

   FirstName VARCHAR(50),

   Salary DECIMAL(10,2),

   SalaryDate DATE

);
```

Step 2: Inserting Sample Data

```
INSERT INTO EmployeeSalaries (EmployeeID, FirstName, Salary, SalaryDate)

VALUES (1, 'Alice', 40000, '2024-01-01'),(2, 'Bob', 45000, '2024-02-01'),

(3, 'Charlie', 50000, '2024-03-01'),(4, 'David', 55000, '2024-04-01'),

(5, 'Emma', 60000, '2024-05-01');
```

Step 3: Applying LAG() Function

```
SELECT EmployeeID, FirstName,Salary, SalaryDate,

LAG(Salary) OVER (ORDER BY SalaryDate) AS PrevSalary FROM EmployeeSalaries;
```

📌 Explanation

- LAG(Salary) **retrieves the previous salary value**.
- ORDER BY SalaryDate ensures that the salaries are compared sequentially by date.

🔷 Sample Output

EmployeeID	FirstName	Salary	SalaryDate	PrevSalary
1	Alice	40000	2024-01-01	NULL
2	Bob	45000	2024-02-01	40000
3	Charlie	50000	2024-03-01	45000
4	David	55000	2024-04-01	50000
5	Emma	60000	2024-05-01	55000

📌 **Notice:** NULL appears for the first row because there is no previous salary.

Step 4: Applying LEAD() Function

Objective: Get the **next salary** of each employee.

```
SELECT EmployeeID, FirstName, Salary,SalaryDate,

LEAD(Salary) OVER (ORDER BY SalaryDate) AS NextSalary

FROM EmployeeSalaries;
```

📌 Explanation

- LEAD(Salary) **retrieves the next salary value**.
- ORDER BY SalaryDate ensures correct order.

🔷 Sample Output

EmployeeID	FirstName	Salary	SalaryDate	NextSalary
1	Alice	40000	2024-01-01	45000
2	Bob	45000	2024-02-01	50000
3	Charlie	50000	2024-03-01	55000
4	David	55000	2024-04-01	60000
5	Emma	60000	2024-05-01	NULL

📌 **Notice:** NULL appears in the last row because there is no next salary.

Step 5: Using LAG() and LEAD() Together

We can **combine both functions** in a single query to compare **previous, current, and next salaries**.

```
SELECT EmployeeID, FirstName, Salary, SalaryDate,

LAG(Salary) OVER (ORDER BY SalaryDate) AS PrevSalary,

LEAD(Salary) OVER (ORDER BY SalaryDate) AS NextSalary    FROM EmployeeSalaries;
```

🔷 Sample Output

EmployeeID	FirstName	Salary	SalaryDate	PrevSalary	NextSalary
1	Alice	40000	2024-01-01	NULL	45000
2	Bob	45000	2024-02-01	40000	50000
3	Charlie	50000	2024-03-01	45000	55000
4	David	55000	2024-04-01	50000	60000
5	Emma	60000	2024-05-01	55000	NULL

📌 Key Insights:

- Each row shows **previous and next salaries** for easy comparison.
- NULL appears for the first row (no previous salary) and last row (no next salary).

Step-by-Step Execution of Percentile & Cumulative Distribution Functions in SQL

We will go through the **entire process**, from **table creation** to **query execution** with PERCENT_RANK(), CUME_DIST(), and NTILE(4).

Percentile & Cumulative Functions

Useful for **percentile analysis, cumulative sums, and moving averages**.

Function	Description
PERCENT_RANK()	Returns **percentile rank** (relative position between 0 and 1).
CUME_DIST()	Returns **cumulative distribution** (fraction of rows ≤ current row).
NTILE(n)	Divides rows into **n equal groups**.

📌 Step 1: Creating the Table

First, we create a table called EmployeeSalaries to store employee salary details.

```
CREATE TABLE EmployeeSalaries (
    EmployeeID INT PRIMARY KEY,
    FirstName VARCHAR(50),
    Salary DECIMAL(10,2)
);
```

📌 Table Structure:

Column	Data Type	Description
EmployeeID	INT (Primary Key)	Unique ID for each employee.
FirstName	VARCHAR(50)	Employee's first name.
Salary	DECIMAL(10,2)	Employee's salary.

📌 Step 2: Inserting Sample Data

We will insert salary records for multiple employees.

```
INSERT INTO EmployeeSalaries (EmployeeID, FirstName, Salary)
VALUES (1, 'Alice', 60000),(2, 'Bob', 50000),(3, 'Charlie', 70000),(4, 'David', 40000),
(5, 'Emma', 65000),(6, 'Frank', 45000),(7, 'Grace', 55000),(8, 'Henry', 52000);
```

📌 Inserted Data:

EmployeeID	FirstName	Salary
1	Alice	60000
2	Bob	50000
3	Charlie	70000
4	David	40000
5	Emma	65000
6	Frank	45000
7	Grace	55000
8	Henry	52000

📌 Step 3: Executing the Query with Window Functions

Now, we use **PERCENT_RANK()**, **CUME_DIST()**, and **NTILE(4)** to analyze salary distribution

```
SELECT EmployeeID, FirstName, Salary,

PERCENT_RANK() OVER (ORDER BY Salary DESC) AS  PercentileRank,

 CUME_DIST()  OVER (ORDER BY Salary DESC) AS  CumulativeDist,

   NTILE(4) OVER (ORDER BY Salary DESC) AS  SalaryQuartile

  FROM EmployeeSalaries;
```

📌 Step 4: Understanding the Output

EmployeeID	FirstName	Salary	PercentileRank	CumulativeDist	SalaryQuartile
3	Charlie	70000	0.0000	0.1250	1
5	Emma	65000	0.1429	0.2500	1
1	Alice	60000	0.2857	0.3750	2
7	Grace	55000	0.4286	0.5000	2
8	Henry	52000	0.5714	0.6250	3
2	Bob	50000	0.7143	0.7500	3
6	Frank	45000	0.8571	0.8750	4
4	David	40000	1.0000	1.0000	4

📌 Step 5: Explanation of Functions

1 `PERCENT_RANK()`

- **Formula**:

$$\frac{(\text{Current Rank - 1})}{(\text{Total Rows - 1})}$$

- Compares how high a salary ranks **relative to others**.
- The highest salary gets `0.0000`, while the lowest gets `1.0000`.

2 CUME_DIST()

- **Formula**:

2 `CUME_DIST()`

- **Formula**:

$$\frac{\text{Number of rows with Salary} \leq \text{current row}}{\text{Total Rows}}$$

- Measures **how many employees earn less than or equal to a given salary**.
- The highest salary gets `0.1250` (1 out of 8), while the lowest gets `1.0000`.

3 NTILE(4)

- Divides employees into **4 equal salary quartiles**.
- **Quartile 1:** Top earners
- **Quartile 4:** Lowest earners

Pivot and Unpivot in SQL

In SQL, **PIVOT** and **UNPIVOT** are powerful operations used to **transform data** and make it more readable, efficient, and manageable. These operations allow us to **manipulate tables** by switching between rows and columns, which can be crucial for **summarizing data**, reporting, and **data analysis**. Understanding how to use **PIVOT** and **UNPIVOT** operators effectively can significantly enhance our data manipulation capabilities.

In this article, we'll explore what **PIVOT** and **UNPIVOT** are, their syntax, key differences, and provide practical examples with explanations to help us master these SQL operations.

What is Pivot in SQL?

The PIVOT operation in SQL allows us to **rotate data from rows to columns**. This can be particularly useful when we need to **summarize**, **aggregate**, or **reorganize** data for reporting purposes. When we apply **PIVOT**, each unique value in a column is turned into its own column, and the data is aggregated based on a specified function, such as **SUM**, **AVG**, COUNT, etc.

Syntax:

```
SELECT (ColumnNames)
FROM (TableName)
PIVOT
(
AggregateFunction   (ColumnToBeAggregated)
FOR   PivotColumn IN  (PivotColumnValues)
) AS (Alias)

//Alias is a temporary name for a table
```

SQL PIVOT diagram

You can use PIVOT to rotate rows in a table by turning row values into multiple columns. The following diagram illustrates what PIVOT can do where we take 4 rows of data and turn this into 1 row with 4 columns. As you can see, the PIVOT process converts rows into columns by pivoting the table.

Col1	Col2
1	3445
2	2334
3	5555
4	5555

Col2	1	2	3	4
Col2	3445	2334	5555	5555

SQL UNPIVOT diagram

On the other hand, unpivot does the opposite and rotates multiple columns into multiple rows.

Col1	1	2	3	4
Col2	3445	2334	5555	5555

Col1	Col2
1	3445
2	2334
3	5555
4	5555

Both, the PIVOT operator and UNPIVOT operator are usually used for reporting purposes when the user requires the information in a different format. However, sometimes, UNPIVOT is used to normalize the output of a query.

For example, imagine a table with data like this:

CustomerID	Phone1	Phone2	Phone3
1	705-491-1111	705-491-1110	NULL
2	613-492-2222	NULL	NULL
3	416-493-3333	416-493-3330	416-493-3339

For a certain report, the desired output is actually as follows (apologies for the messy arrows):

CustomerID	Phone1	Phone2	Phone3
1	705-491-1111	705-491-1110	NULL
2	613-492-2222	NULL	NULL
3	416-493-3333	416-493-3330	416-493-3339

CustomerID	Phone
1	705-491-1111
1	705-491-1110
2	613-492-2222
3	416-493-3333
3	416-493-3330
3	416-493-3339

Another scenario is when we want to export data and we need it in a different format and table structure. In those scenarios, we sometimes need to PIVOT or UNPIVOT data.

Pivot and Unpivot in SQL

*In SQL, **PIVOT** and **UNPIVOT** are powerful operations used to **transform data** and make it more readable, efficient, and manageable. These operations allow us to **manipulate tables** by switching between rows and columns, which can be crucial for **summarizing data**, reporting, and **data analysis**. Understanding how to use **PIVOT** and **UNPIVOT** operators effectively can significantly enhance our data manipulation capabilities.*

What is Pivot in SQL?

*The PIVOT operation in SQL allows us to **rotate data from rows to columns**. This can be particularly useful when we need to **summarize**, **aggregate**, or **reorganize** data for reporting purposes. When we apply **PIVOT**, each unique value in a column is turned into its own column, and the data is aggregated based on a specified function, such as **SUM**, **AVG**, COUNT, etc.*

Example of Pivot Operation:

*Let's say we have a sample table **geeksforgeeks** that contains data about **various courses**, their **categories**, and their prices. We can use the **PIVOT** operator to summarize the total price for each course category.*

CourseName	***CourseCategory***	***Price***
C	*PROGRAMMING*	*5000*
JAVA	*PROGRAMMING*	*6000*
PYTHON	*PROGRAMMING*	*8000*
PLACEMENT 100	*INTERVIEWPREPARATION*	*5000*

Query:

```
SELECT CourseName, PROGRAMMING, INTERVIEWPREPARATION
FROM geeksforgeeks
PIVOT
(
SUM(Price)
FOR CourseCategory IN (PROGRAMMING, INTERVIEWPREPARATION )
) AS PivotTable
```

Output

CourseName	PROGRAMMING	INTERVIEWPREPARATION
C	5000	NULL
JAVA	6000	NULL
PLACEMENT 100	NULL	5000
PYTHON	8000	NULL

Output of Pivot operation

***Explanation**:*

*The **PIVOT** operation reorganizes the data, transforming the **CourseCategory** values into columns, while the **SUM(Price)** function aggregates the prices based on these categories. Notice that the original rows are now represented as columns.*

```
SELECT (ColumnNames) FROM (TableName)
PIVOT
(
AggregateFunction(ColumnToBeAggregated)
FOR PivotColumn IN (PivotColumnValues)
) AS (Alias) //Alias is a temporary name for a table
```

Use of Aggregation Function in PIVOT and UNPIVOT in SQL

Aggregation functions are essential in **PIVOT** because they determine how multiple row values are combined into a single column value. **UNPIVOT**, however, does not require an aggregation function because it converts column values back into rows.

Syntax:

***Note**: MySQL Does Not Support PIVOT Directly Unlike SQL Server, MySQL does not have a built-in PIVOT function. Instead, you can achieve the same result using CASE statements with GROUP BY.*

```
create database pivot;

use pivot

CREATE TABLE geeksforgeeks (

    CourseName VARCHAR(100),

    CourseCategory VARCHAR(50),

    Price INT
```

```
);

INSERT INTO geeksforgeeks (CourseName, CourseCategory, Price) VALUES
('React JS Course', 'PROGRAMMING', 6500),
('Django Full Stack', 'PROGRAMMING', 7200),
('C++ for Beginners', 'PROGRAMMING', 3500),
('Node.js Backend', 'PROGRAMMING', 7800),
-- New Interview Preparation Courses

('Behavioral Interview Tips', 'INTERVIEWPREPARATION', 4500),
('Resume Writing Workshop', 'INTERVIEWPREPARATION', 3200),
('Mock Coding Interviews', 'INTERVIEWPREPARATION', 7000),
('Advanced DSA Problems', 'INTERVIEWPREPARATION', 9000),

-- New Data Science & Machine Learning Courses
('Data Science with Python', 'DATASCIENCE', 8500),
('Machine Learning A-Z', 'DATASCIENCE', 9200),
('Deep Learning with TensorFlow', 'DATASCIENCE', 10500),
('Big Data with Hadoop', 'DATASCIENCE', 8700),

-- New DevOps Courses
('Docker and Kubernetes', 'DEVOPS', 7800),
('AWS Cloud Practitioner', 'DEVOPS', 8200),
('CI/CD with Jenkins', 'DEVOPS', 7500),
```

```
('Terraform for Infrastructure', 'DEVOPS', 7900);

select * from geeksforgeeks ;

-------------------------------------------------------------------------------------------------------------------------------

-- SELECT CourseName, PROGRAMMING, INTERVIEWPREPARATION

-- FROM geeksforgeeks

-- PIVOT (

-- SUM(Price)

-- FOR CourseCategory IN (PROGRAMMING, INTERVIEWPREPARATION )

-- ) AS PivotTable

-------------------------------------------------------------------------------------------------------------------------------

SELECT  CourseName,

 SUM(CASE WHEN CourseCategory = 'PROGRAMMING' THEN Price ELSE NULL END) AS PROGRAMMING,

 SUM(CASE WHEN CourseCategory = 'INTERVIEWPREPARATION' THEN Price ELSE NULL END) AS INTERVIEWPREPARATION

FROM geeksforgeeks  GROUP BY CourseName;

-------------------------------------------------------------------------------------------------------------------------------

SELECT  CourseName,

  -- Total price using SUM

 AVG(CASE WHEN CourseCategory = 'PROGRAMMING' THEN Price ELSE NULL END) AS Avg_Programming_Price,

 AVG(CASE WHEN CourseCategory = 'INTERVIEWPREPARATION' THEN Price ELSE NULL END) AS Avg_Interview_Prep_Price

FROM  geeksforgeeks  GROUP BY CourseName;

-------------------------------------------------------------------------------------------------------------------------------

SELECT CourseName,
```

```
   -- Maximum price using MAX

MAX(CASE WHEN CourseCategory = 'PROGRAMMING' THEN Price ELSE NULL END)

AS Max_Programming_Price,

MAX(CASE WHEN CourseCategory = 'INTERVIEWPREPARATION' THEN Price ELSE NULL END)
AS Max_Interview_Prep_Price

FROM  geeksforgeeks GROUP BY CourseName;

SELECT CourseName,

-- Minimum price using MIN

MIN(CASE WHEN CourseCategory = 'PROGRAMMING' THEN Price ELSE NULL END)

AS Min_Programming_Price,

MIN(CASE WHEN CourseCategory = 'INTERVIEWPREPARATION' THEN Price ELSE NULL END)
AS Min_Interview_Prep_Price

FROM  geeksforgeeks  GROUP BY CourseName;

-----------------------------------------------------------------------------------------------------------------------

SELECT CourseName,

-- Count of courses using COUNT

 COUNT(CASE WHEN CourseCategory = 'PROGRAMMING' THEN 1 ELSE NULL END) AS
Count_Programming_Courses,

 COUNT(CASE WHEN CourseCategory = 'INTERVIEWPREPARATION' THEN 1 ELSE NULL END) AS
Count_Interview_Prep_Courses

FROM  geeksforgeeks GROUP BY CourseName;

-----------------------------------------------------------------------------------------------------------------------

SELECT  CourseName,

   -- Total price using SUM

 SUM(CASE WHEN CourseCategory = 'PROGRAMMING' THEN Price ELSE 0 END) AS
Total_Programming_Price,
```

```
 SUM(CASE WHEN CourseCategory = 'INTERVIEWPREPARATION' THEN Price ELSE 0 END) AS Total_Interview_Prep_Price,

-- Average price using AVG

 AVG(CASE WHEN CourseCategory = 'PROGRAMMING' THEN Price ELSE NULL END) AS Avg_Programming_Price,

 AVG(CASE WHEN CourseCategory = 'INTERVIEWPREPARATION' THEN Price ELSE NULL END) AS Avg_Interview_Prep_Price,

-- Maximum price using MAX

 MAX(CASE WHEN CourseCategory = 'PROGRAMMING' THEN Price ELSE NULL END) AS Max_Programming_Price,

 MAX(CASE WHEN CourseCategory = 'INTERVIEWPREPARATION' THEN Price ELSE NULL END) AS Max_Interview_Prep_Price,

-- Minimum price using MIN

MIN(CASE WHEN CourseCategory = 'PROGRAMMING' THEN Price ELSE NULL END) AS Min_Programming_Price,

MIN(CASE WHEN CourseCategory = 'INTERVIEWPREPARATION' THEN Price ELSE NULL END) AS Min_Interview_Prep_Price,

 -- Count of courses using COUNT

 COUNT(CASE WHEN CourseCategory = 'PROGRAMMING' THEN 1 ELSE NULL END) AS Count_Programming_Courses,

COUNT(CASE WHEN CourseCategory = 'INTERVIEWPREPARATION' THEN 1 ELSE NULL END) AS Count_Interview_Prep_Courses

FROM geeksforgeeks GROUP BY CourseName;
```

*The **PIVOT(only support in SQL server)** operation reorganizes the data, transforming the **CourseCategory** values into columns, while the **SUM(Price)** function aggregates the prices based on these categories. Notice that the original rows are now represented as columns.*

1. Pivot Example: Converting Rows to Columns

Scenario:

We have a StudentMarks table that stores marks for students in different subjects.

Student	*Subject*	*Marks*
Alice	*Math*	*85*
Alice	*Science*	*90*
Bob	*Math*	*78*
Bob	*Science*	*88*

Goal: Convert subjects into columns.

SQL Query (Pivot)

```
SELECT * FROM (

   SELECT Student, Subject, Marks

   FROM StudentMarks

) AS SourceTable

PIVOT (    MAX(Marks) FOR Subject IN ([Math], [Science])     ) AS PivotTable;
```

Since **MySQL does not support PIVOT**, we can achieve the same result using **conditional aggregation (CASE WHEN)**.

✅ **Corrected Query for MySQL:**

```
SELECT Student,

   MAX(CASE WHEN Subject = 'Math' THEN Marks ELSE NULL END) AS Math,

   MAX(CASE WHEN Subject = 'Science' THEN Marks ELSE NULL END) AS Science

 FROM StudentMarks GROUP BY Student;
```

Result:

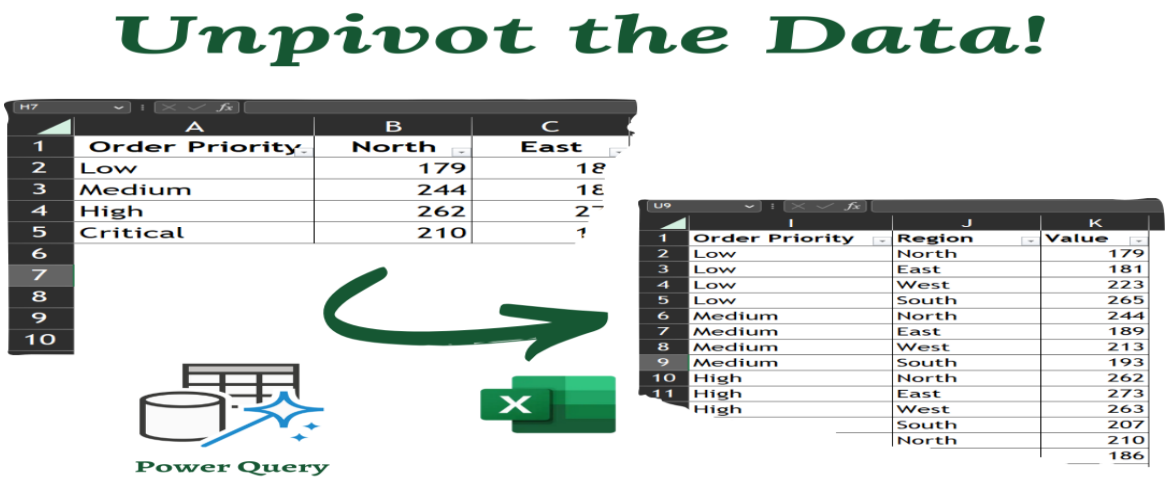
Student	*Math*	*Science*
Alice	*85*	*90*
Bob	*78*	*88*

What is Unpivot in SQL?

The UNPIVOT operation in SQL is the reverse of PIVOT. It allows us to convert columns back into rows, which can be useful when we need to normalize or restructure data after performing a pivot operation. UNPIVOT helps in data analysis where row-based operations are required for analysis.

Syntax:

```
SELECT (ColumnNames)
FROM (TableName)
UNPIVOT
(AggregateFunction(ColumnToBeAggregated)
FOR PivotColumn IN (PivotColumnValues)
) AS (Alias)
```



Example of Unpivot Operation:

Now, we use the same table "geeksforgeeks" created in the above example and apply the UNPIVOT operation to reverse the PIVOT operation above and retrieve the original data.

Query:

```
SELECT CourseName, CourseCategory, Price
FROM
(
   SELECT CourseName, PROGRAMMING, INTERVIEWPREPARATION
   FROM geeksforgeeks
   PIVOT
   (
      SUM(Price)
      FOR CourseCategory IN (PROGRAMMING, INTERVIEWPREPARATION)
   ) AS PivotTable
) AS P
UNPIVOT
(
   Price FOR CourseCategory IN (PROGRAMMING, INTERVIEWPREPARATION)
) AS UnpivotTable;
```

Output

CourseName	CourseCategory	Price
C	PROGRAMMING	5000
JAVA	PROGRAMMING	6000
PLACEMENT 100	INTERVIEWPREPARATION	5000
PYTHON	PROGRAMMING	8000

<u>Output of Unpivot operation</u>

The UNPIVOT operation transforms the column-based data (PROGRAMMING and INTERVIEWPREPARATION) back into rows, allowing us to return to the original structure. It essentially reverts the result set from the PIVOT operation.

When to Use Pivot and Unpivot in SQL

- ***PIVOT**: When you need to summarize or aggregate data and display it in a columnar format, typically for reporting purposes.*
- ***UNPIVOT**: When you need to convert column data back to rows, often after performing a **PIVOT** operation, for normalization or further analysis.*

PIVOT and UNPIVOT in Power Query (Excel & Power BI)

*Power Query provides **PIVOT** and **UNPIVOT** functions to reshape data efficiently.*

◈ PIVOT in Power Query

📌 What It Does:

- Converts **unique values** from a column into **new column headers**.
- Aggregates values where necessary.

✅ Example - PIVOT Transformation:

Input Data (Sales Data)

Product	Month	Sales
Laptop	Jan	5000
Laptop	Feb	6000
Phone	Jan	7000
Phone	Feb	7500

🎯 **Goal:** Convert Month values into separate columns.

PIVOTED Output (After Pivoting "Month" Column)

Product	Jan	Feb
Laptop	5000	6000
Phone	7000	7500

🛠 Steps to PIVOT in Power Query:

1. Load the data into Power Query (Excel/Power BI).
2. Select the **column** to pivot (Month).
3. Click **Transform → Pivot Column**.
4. Choose **Values Column** (Sales).
5. Select **Aggregation** (default is SUM).
6. Click **OK** to apply.

🔷 UNPIVOT in Power Query

📌 What It Does:

- Converts **multiple columns into rows**.
- Useful when data is stored in a **wide format** and needs normalization.

✅ Example - UNPIVOT Transformation:

Input Data (Wide Format)

Product	Jan	Feb
Laptop	5000	6000
Phone	7000	7500

🎯 **Goal:** Convert Jan & Feb columns into a **single column** named Month.

UNPIVOTED Output (After Unpivoting "Jan" & "Feb" Columns)

Product	Month	Sales
Laptop	Jan	5000
Laptop	Feb	6000
Phone	Jan	7000
Phone	Feb	7500

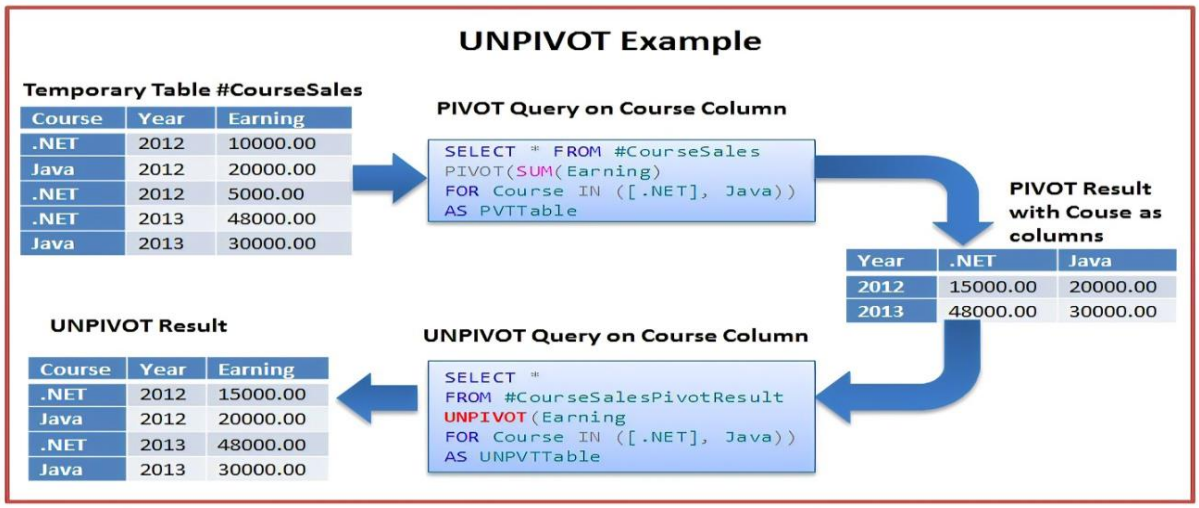
🛠 Steps to UNPIVOT in Power Query:

1. Load the **wide data** into Power Query.
2. Select **multiple columns** (e.g., Jan, Feb).

3. Click **Transform → Unpivot Columns**.
4. Rename the generated **Attribute** column to Month.
5. Rename **Value** column to Sales.

📌 Summary of PIVOT vs. UNPIVOT

Feature	PIVOT	UNPIVOT
What it does	Converts rows into columns	Converts columns into rows
Use case	Makes data **wider** (summary tables)	Makes data **longer** (normalization)
Common Application	Monthly sales reports, survey results	Converting multi-period data for analysis



Example-1:

CREATE TABLE StudentMarks_Pivot (

Student VARCHAR(50),

Math INT,

Science INT,

English INT

);

*select * from StudentMarks_Pivot;*

```
INSERT INTO StudentMarks_Pivot (Student, Math, Science, English)

VALUES

('Alice', 85, 90, 88),

('Bob', 78, 88, 80),

('Charlie', 92, 95, 89),

('David', 75, 85, 82),

('Eve', 88, 91, 87),

('Frank', 79, 83, 81),

('Grace', 94, 97, 90),

('Hannah', 80, 89, 85), ('Isaac', 82, 84, 78), ('Jack', 90, 92, 88);

-- SELECT Student, Subject, Marks

-- FROM

--     (SELECT Student, Math, Science, English FROM StudentMarks_Pivot) AS
SourceTable

-- UNPIVOT

--     (Marks FOR Subject IN (Math, Science, English))

-- AS UnpivotTable;

-- MySQL does not support UNPIVOT, but you can achieve the same result using UNION
ALL:

SELECT Student, 'Math' AS Subject, Math AS Marks FROM StudentMarks_Pivot

UNION ALL

SELECT Student, 'Science' AS Subject, Science AS Marks FROM StudentMarks_Pivot

UNION ALL

SELECT Student, 'English' AS Subject, English AS Marks FROM StudentMarks_Pivot;
```

-- Variable Type Description

-- Student Column The student's name (taken directly from StudentMarks_Pivot)

-- Math Column Math marks for each student (from StudentMarks_Pivot)

-- Science Column Science marks for each student (from StudentMarks_Pivot)

-- English Column English marks for each student (from StudentMarks_Pivot)

-- 'Math' Constant (String) A fixed value assigned as the Subject in the first SELECT

-- 'Science' Constant (String) A fixed value assigned as the Subject in the second SELECT

-- 'English' Constant (String) A fixed value assigned as the Subject in the third SELECT

==========

-- 1)First SELECT Block:

-- SELECT Student, 'Math' AS Subject, Math AS Marks FROM StudentMarks_Pivot

-- Selects Student name

-- Assigns 'Math' as the Subject (a constant value)

-- Retrieves Math marks from StudentMarks_Pivot

==========
2)Second SELECT Block (Using UNION ALL):

-- UNION ALL

-- SELECT Student, 'Science' AS Subject, Science AS Marks FROM StudentMarks_Pivot

-- Selects the same Student column

-- Assigns 'Science' as the Subject (a constant value)

-- Retrieves Science marks

==========

-- 3)Third SELECT Block (Using UNION ALL):

-- UNION ALL

-- SELECT Student, 'English' AS Subject, English AS Marks FROM StudentMarks_Pivot;

-- Again selects Student

-- Assigns 'English' as the Subject

-- Retrieves English marks

==========

-- Fixed values ('Math', 'Science', 'English') are used to convert columns into rows.

-- Columns (Student, Math, Science, English) are used to fetch data from the table.

-- UNION ALL is used to combine the results into a single output.

==========

-- unpivoted supported query in SQL

-- ----------

-- SELECT Student, Subject, Marks

-- FROM

```
--(SELECT Student, Math, Science, English FROM StudentMarks_Pivot) AS SourceTable
--  UNPIVOT
--     (Marks FOR Subject IN (Math, Science, English)) AS UnpivotTable;
```

Example-2:

```
CREATE TABLE CourseEarnings (
    Course VARCHAR(50),
    Year_2022 INT,
    Year_2023 INT,
    Year_2024 INT
);

INSERT INTO CourseEarnings (Course, Year_2022, Year_2023, Year_2024) VALUES
('Python Basics', 5000, 7000, 8500),
('Java Masterclass', 7000, 9500, 12000),
('Data Science', 8000, 11000, 13000),
('SQL Bootcamp', 4500, 6000, 7500),
('Web Development', 9000, 10500, 12500),
('AI & ML', 10000, 13000, 15000),
('Cyber Security', 7500, 9200, 11000),
```

('Cloud Computing', 8800, 10200, 11800),

('Blockchain Basics', 7200, 8500, 9800),

('Digital Marketing', 6800, 8200, 9400);

-- Step 3: Apply UNPIVOT (SQL Server)

-- To convert years from columns into rows, we apply UNPIVOT:

==========

-- SELECT Course, Year, Earnings

-- FROM

-- (SELECT Course, Year_2022, Year_2023, Year_2024 FROM CourseEarnings) AS SourceTable

-- UNPIVOT

-- (Earnings FOR Year IN (Year_2022, Year_2023, Year_2024))

-- AS UnpivotTable;

*SELECT * from CourseEarnings;*

-- Step 3: Apply UNPIVOT (SQL Server)

-- To convert years from columns into rows, we apply UNPIVOT:

==========

-- To convert years from columns into rows, we apply UNPIVOT:

-- SELECT Course, Year, Earnings

-- FROM

-- (SELECT Course, Year_2022, Year_2023, Year_2024 FROM CourseEarnings) AS SourceTable

-- UNPIVOT

-- (Earnings FOR Year IN (Year_2022, Year_2023, Year_2024))

-- AS UnpivotTable

==========

-- Alternative for MySQL (Since UNPIVOT is Not Supported)

-- If you're using MySQL, use UNION ALL:

SELECT course,'Year_2022' as Year,Year_2022 as Earning from CourseEarnings

UNION ALL

SELECT course,'Year_2023' as Year,Year_2023 as Earning from CourseEarnings

UNION All

SELECT course,'Year_2024' as Year,Year_2024 as Earning from CourseEarnings

FAQs:

What is the difference between pivot and unpivot in SQL?

In SQL, PIVOT transforms rows into columns, summarizing data, while UNPIVOT converts columns back into rows, useful for data normalization.

What is a pivot query in SQL?

A PIVOT query in SQL rotates data from rows to columns, typically used to aggregate data and create more readable reports.

What is the difference between pivot and unpivot in Power Query?

In Power Query, PIVOT turns unique values from a column into multiple columns, while UNPIVOT does the reverse, converting columns into rows